

IBS: Bezpečnost a počítačové sítě

Cryptography, PKI, TLS/SSL

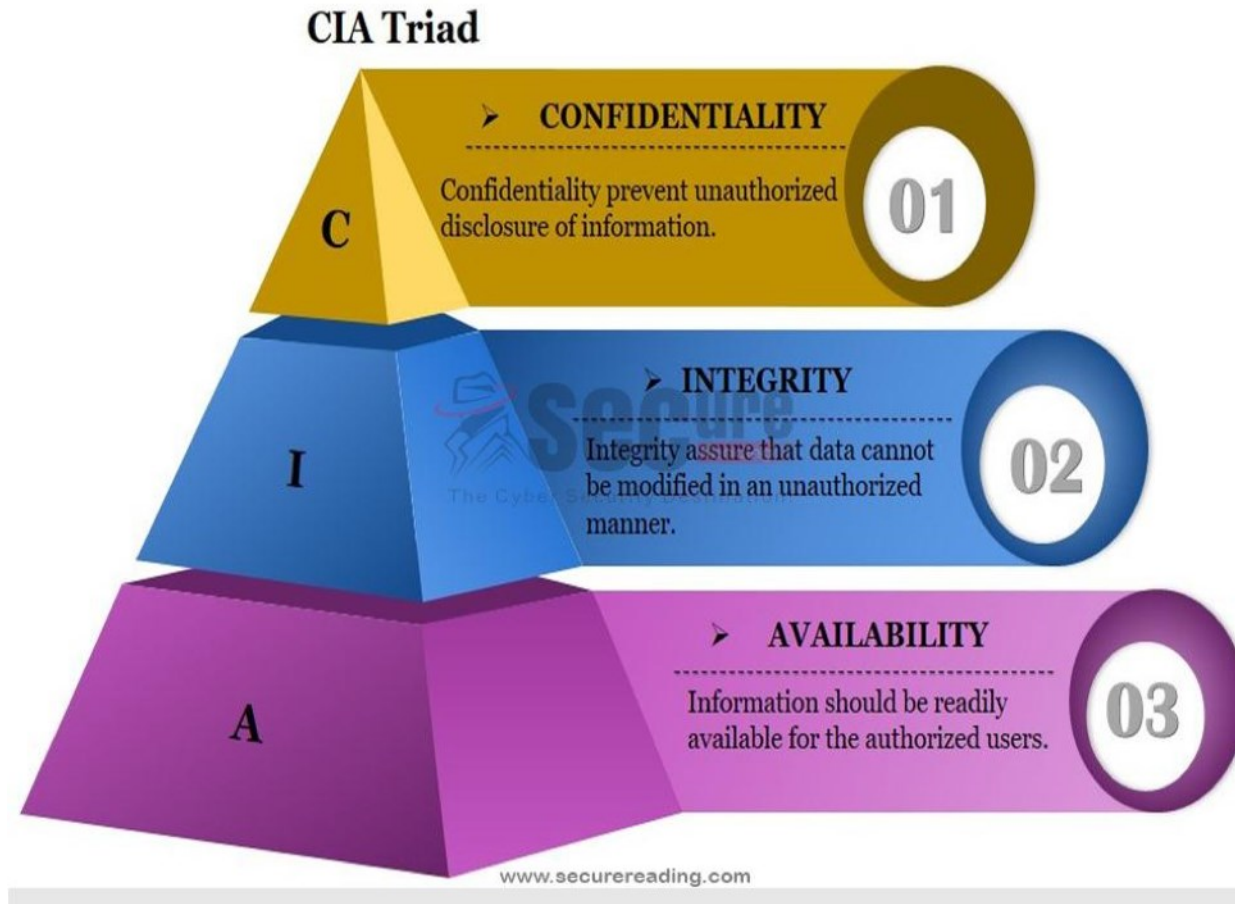
Brno University of Technology, Faculty of Information Technology
Božetěchova 1/2, 612 66 Brno - Královo Pole
Vladimír Veselý, veselyv@fit.vut.cz



17th February 2025

CRYPTOLOGY

- The four elements of securing communications are:
 - 1) Data **Integrity** - Guarantees that the message was not altered.
 - 2) Origin **Authentication** - Guarantees that the message is not a forgery and it actually comes from whom it states.
 - 3) Data **Confidentiality** - Guarantees that only authorized users can read the message.
 - 4) Data **Non-Repudiation** - Guarantees that the sender cannot repudiate, or refute, the validity of a message sent.



- <https://securereading.com/infobasics-basic-concept-information-security/>



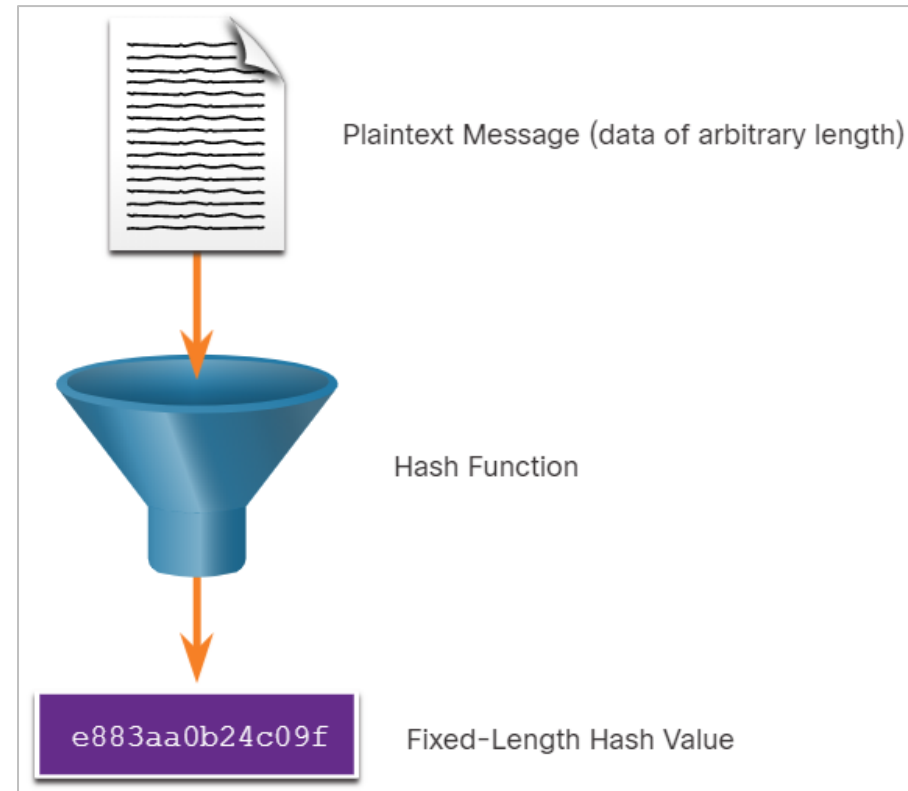
- <https://hackingconcept.wordpress.com/2016/03/08/elements-of-information-security/>
- <https://www.youtube.com/watch?v=6m7mZNvM1Jo>



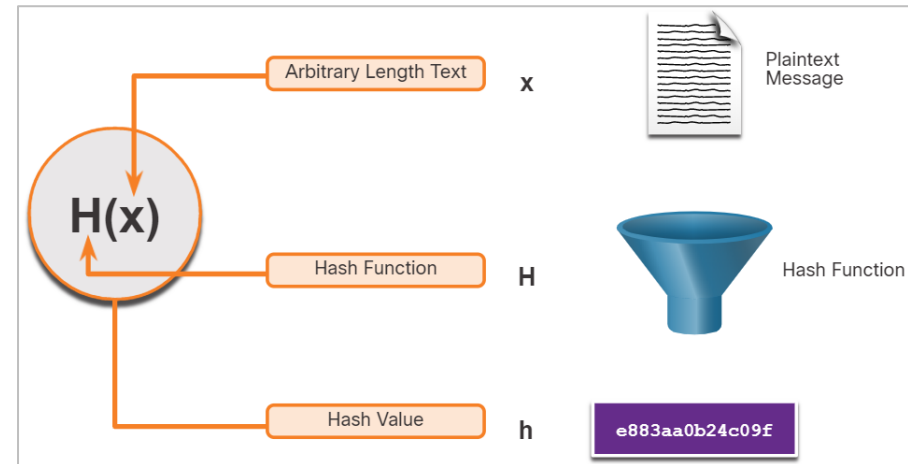
- **Confidentiality:** The limits on who has access to information
- **Integrity:** Whether the information is in its intended state
- **Availability:** Whether the information can be accessed in a timely manner
- **Authenticity:** The proper attribution of the person who created the information
- **Utility:** The usefulness of the information
- **Possession or control:** The physical state where the information is maintained

- <https://mydigitalpublication.com/publication/?i=41813&p=12&pp=1&view=issueViewer>

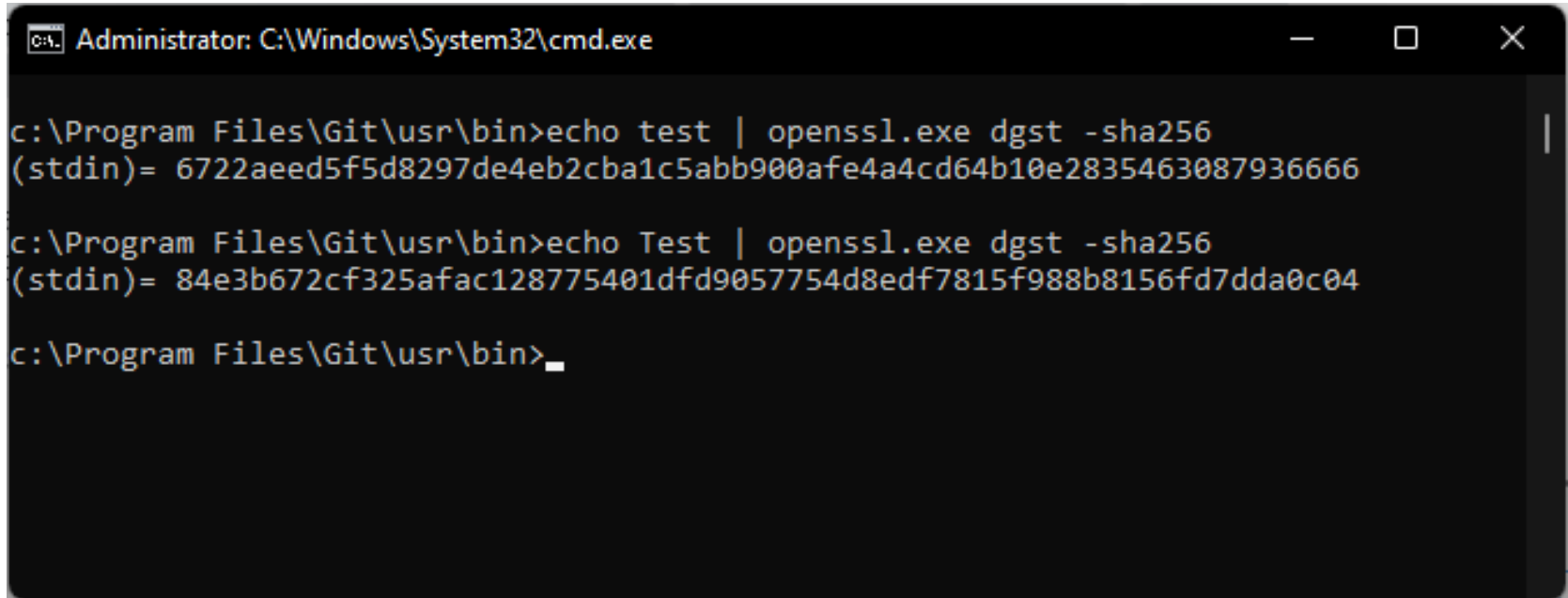
- **Hashes are used to verify and ensure data integrity**
- Hashing is based on a one-way mathematical function that is relatively easy to compute, but significantly harder to reverse.
- A hash function takes a variable block of binary data, called the message, and produces a fixed-length, condensed representation, called the hash.
- The resulting hash is also sometimes called the message digest, digest, or digital fingerprint.
- **With hash functions, it is computationally infeasible for two different sets of data to come up with the same hash output.**
- Every time the data is changed or altered, the hash value also changes.



- Mathematically, the equation $h = H(x)$ is used to explain how a hash algorithm operates.
- As shown in the figure, a hash function H takes an input x and returns a fixed-size string hash value h .
- A cryptographic hash function should have the following properties:
 - The input can be any length.
 - The output has a fixed length.
 - $H(x)$ is relatively easy to compute for given x .
 - $H(x)$ is one way and not reversible.
 - $H(x)$ is collision free, meaning that two different input values will result in different hash values.



Algorithm	Description
MD5 (Message Digest Algorithm 5)	Produces a 128-bit hash value. It was commonly used for checksums and digital signatures. However, it is now considered cryptographically broken and unsuitable for further use due to collision vulnerabilities, where two different inputs can produce the same hash.
SHA-1 (Secure Hash Algorithm 1)	Produces a 160-bit hash value. Previously used in digital signatures and certificates. However, due to successful collision attacks, it is considered deprecated for security purposes and has been replaced by more secure SHA-2 algorithms.
SHA-256 (Secure Hash Algorithm 256-bit)	Part of the SHA-2 family, producing a 256-bit hash value. It is widely used in blockchain technology (e.g., Bitcoin), SSL/TLS certificates, and secure password hashing. SHA-256 offers strong collision resistance.
SHA-512 (Secure Hash Algorithm 512-bit)	Another member of the SHA-2 family, producing a 512-bit hash value. It is suitable for high-security applications such as digital certificates, digital signatures, and data integrity verification due to its longer output length.
RIPEMD-160 (RACE Integrity Primitives Evaluation Message Digest)	A cryptographic hash function producing a 160-bit hash value. It was designed to provide a secure alternative to MD4 and is used in some cryptocurrency protocols, such as Bitcoin for address generation.
BLAKE2	Produces a 256-bit or 512-bit hash value (depending on the variant: BLAKE2s for 256-bit and BLAKE2b for 512-bit). A fast and secure cryptographic hash function designed as an alternative to MD5 and SHA-2. It provides better security than MD5 and similar speed to MD5 but with fewer vulnerabilities. It is commonly used in cryptographic applications, digital signatures, and file integrity verification.
Whirlpool	A cryptographic hash function producing a 512-bit hash value, based on a modified version of the AES block cipher. It is recognized for its strength and is used in applications requiring data integrity, such as digital signatures and secure file storage.
CRC32 (Cyclic Redundancy Check 32-bit)	Produces a 32-bit hash value. A non-cryptographic hash function commonly used for error-checking in data transmission and storage. While it ensures data integrity, it is not suitable for cryptographic purposes due to its susceptibility to intentional collisions.



```
Administrator: C:\Windows\System32\cmd.exe

c:\Program Files\Git\usr\bin>echo test | openssl.exe dgst -sha256
(stdin)= 6722aeed5f5d8297de4eb2cba1c5abb900afe4a4cd64b10e2835463087936666

c:\Program Files\Git\usr\bin>echo Test | openssl.exe dgst -sha256
(stdin)= 84e3b672cf325afac128775401dfd9057754d8edf7815f988b8156fd7dda0c04

c:\Program Files\Git\usr\bin>_
```

- https://www.tools4noobs.com/online_tools/hash/

Calculates the hash of string using various algorithms.

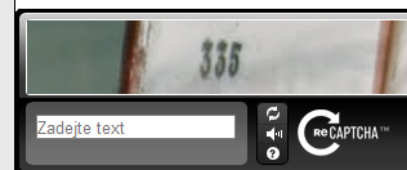
Algorithm: md5

Hash this!

Result: 1f3870be274f6c49b3e31a0c6728957f

- <https://crackstation.net/>

Enter up to 20 non-salted hashes, one per line:



Zadejte text

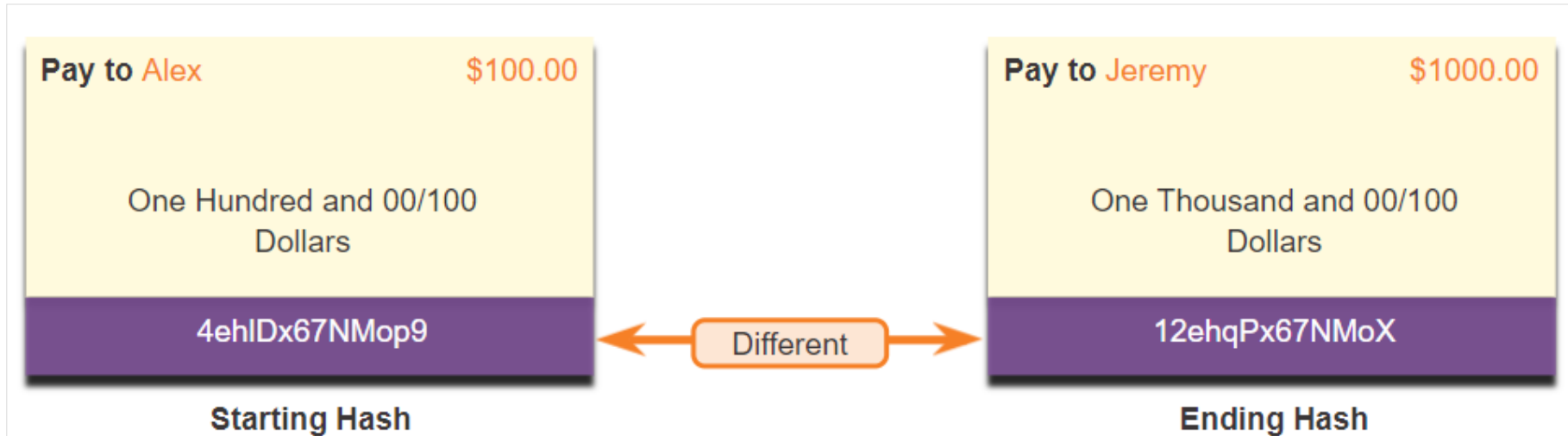


Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1(bin))

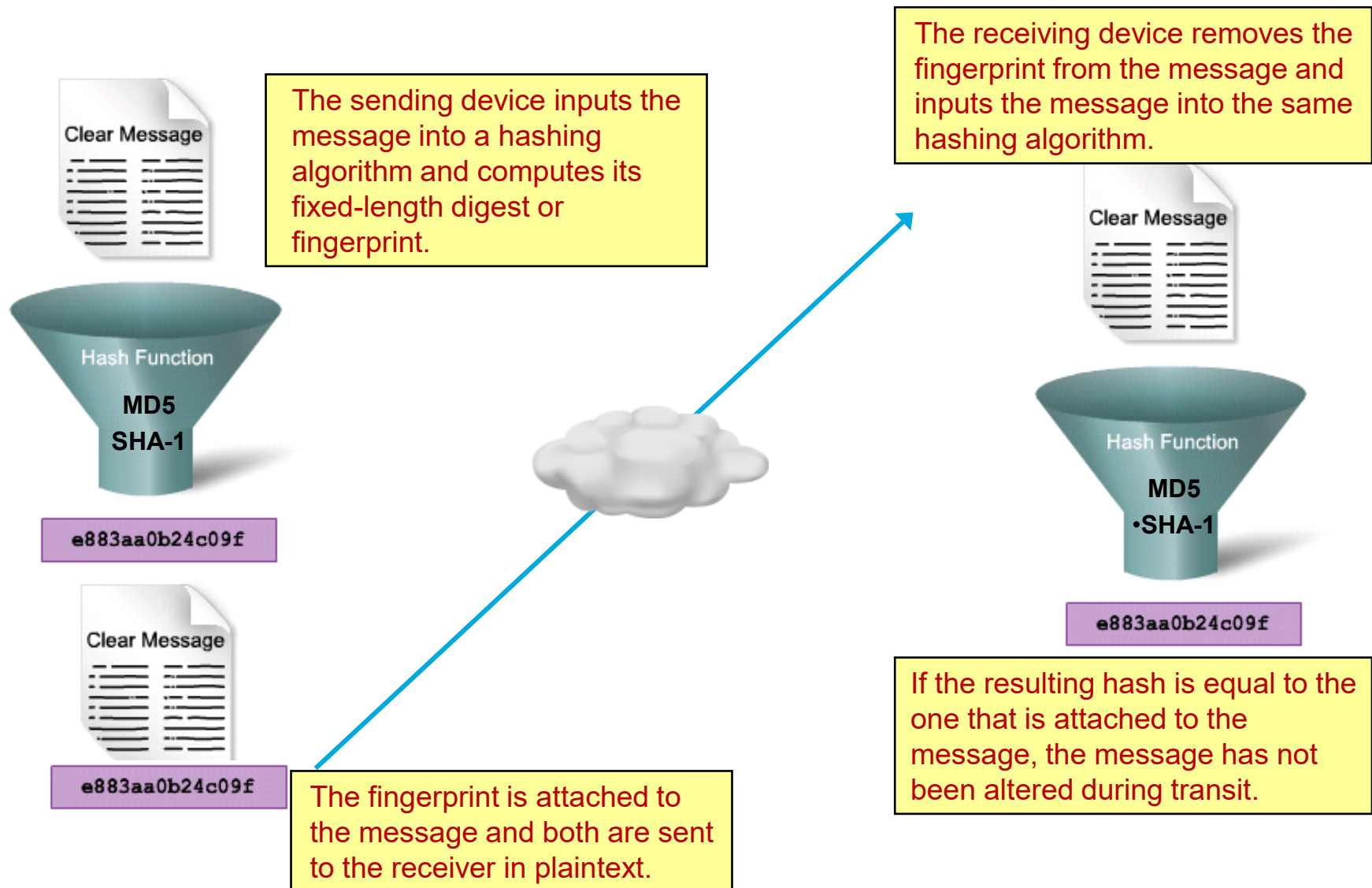
Hash	Type	Result
1f3870be274f6c49b3e31a0c6728957f	md5	apple

Color Codes: Green Exact match, Yellow Partial match, Red Not found.

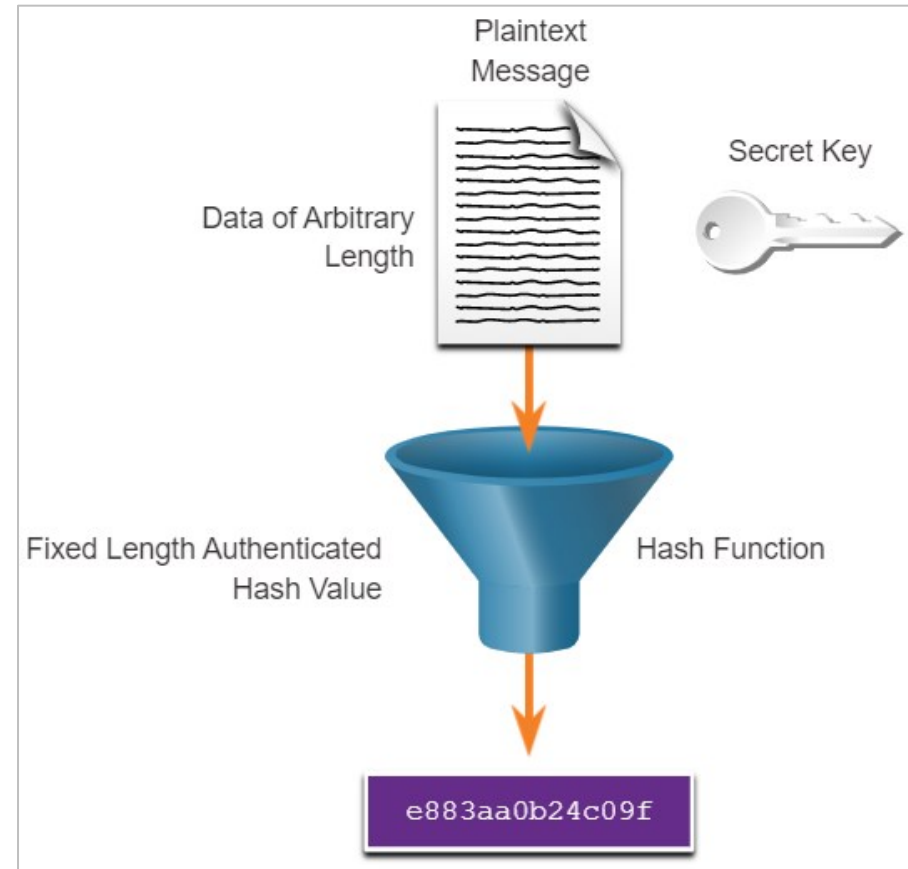


The sender wants to ensure that the message is not altered on its way to the receiver.

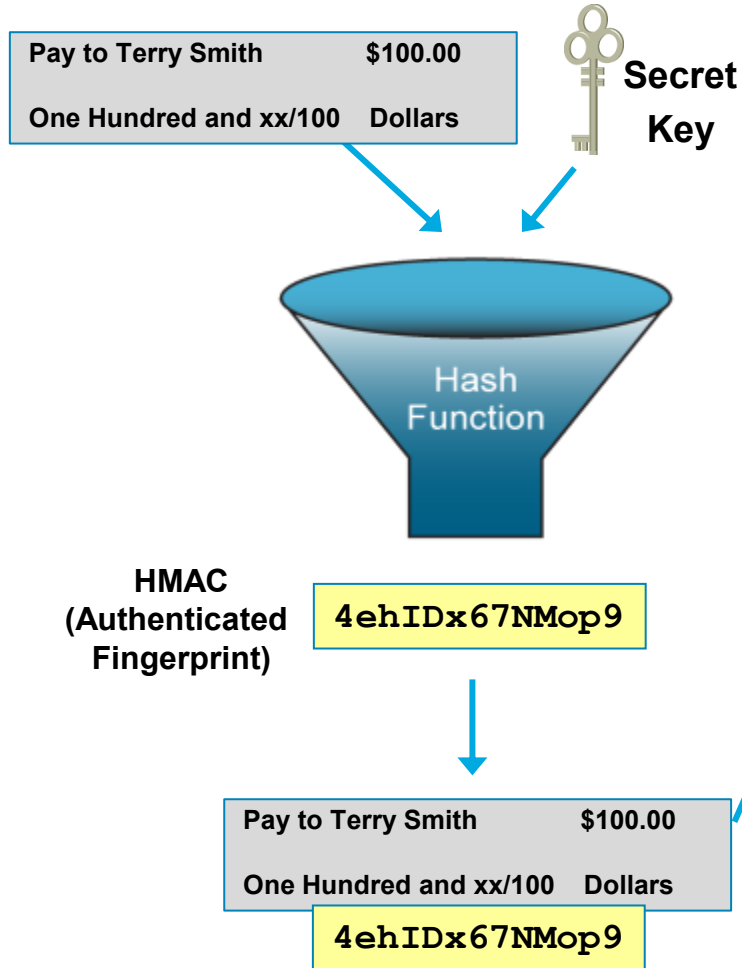
1. The sending device inputs the message into a hashing algorithm and computes its fixed-length hash of 4ehIDx67NMop9.
2. This hash is then attached to the message and sent to the receiver. Both the message and the hash are in plaintext.
3. The receiving device removes the hash from the message and inputs the message into the same hashing algorithm. If the computed hash is equal to the one that is attached to the message, the message has not been altered during transit. If the hashes are not equal, as shown in the figure, then the integrity of the message can no longer be trusted.



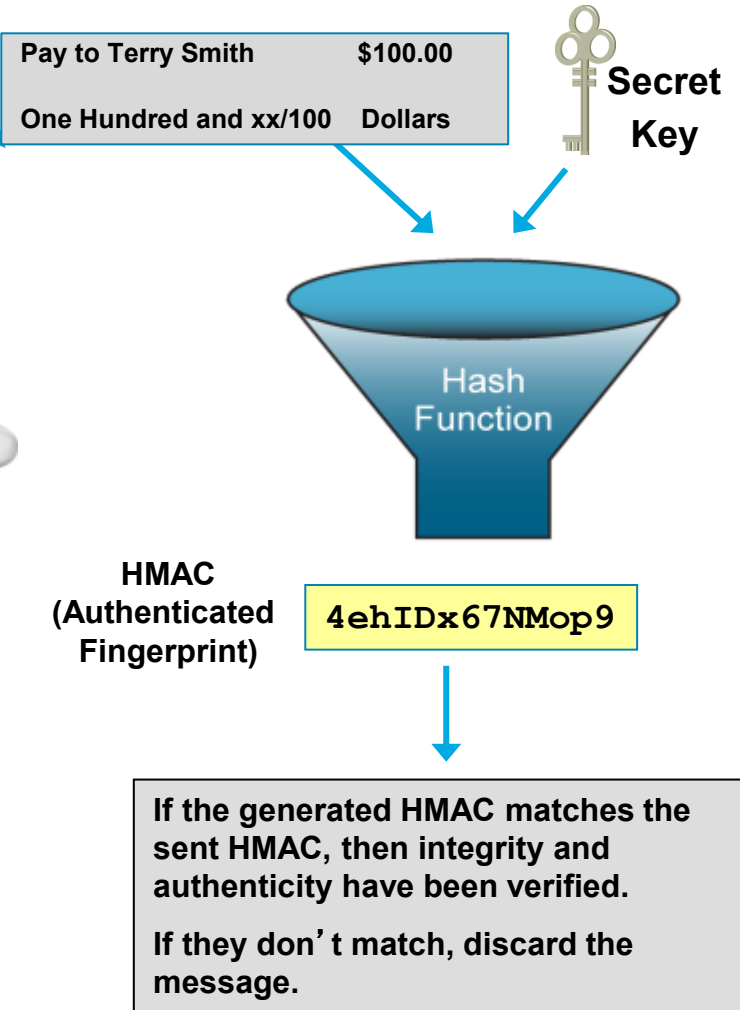
- **HMAC (or KMAC) is a message authentication code (MAC) that is calculated using a hash function and a secret key**
 - Cryptographic strength of the underlying hash function.
 - Size and quality of the key.
 - Size of the hash output length in bits.
- Authenticity is guaranteed because only the sender and the receiver know the secret key.
 - Only they can compute the digest of an HMAC function.
 - This characteristic defeats man-in-the-middle attacks and provides authentication of the data origin.



Data



Received Data



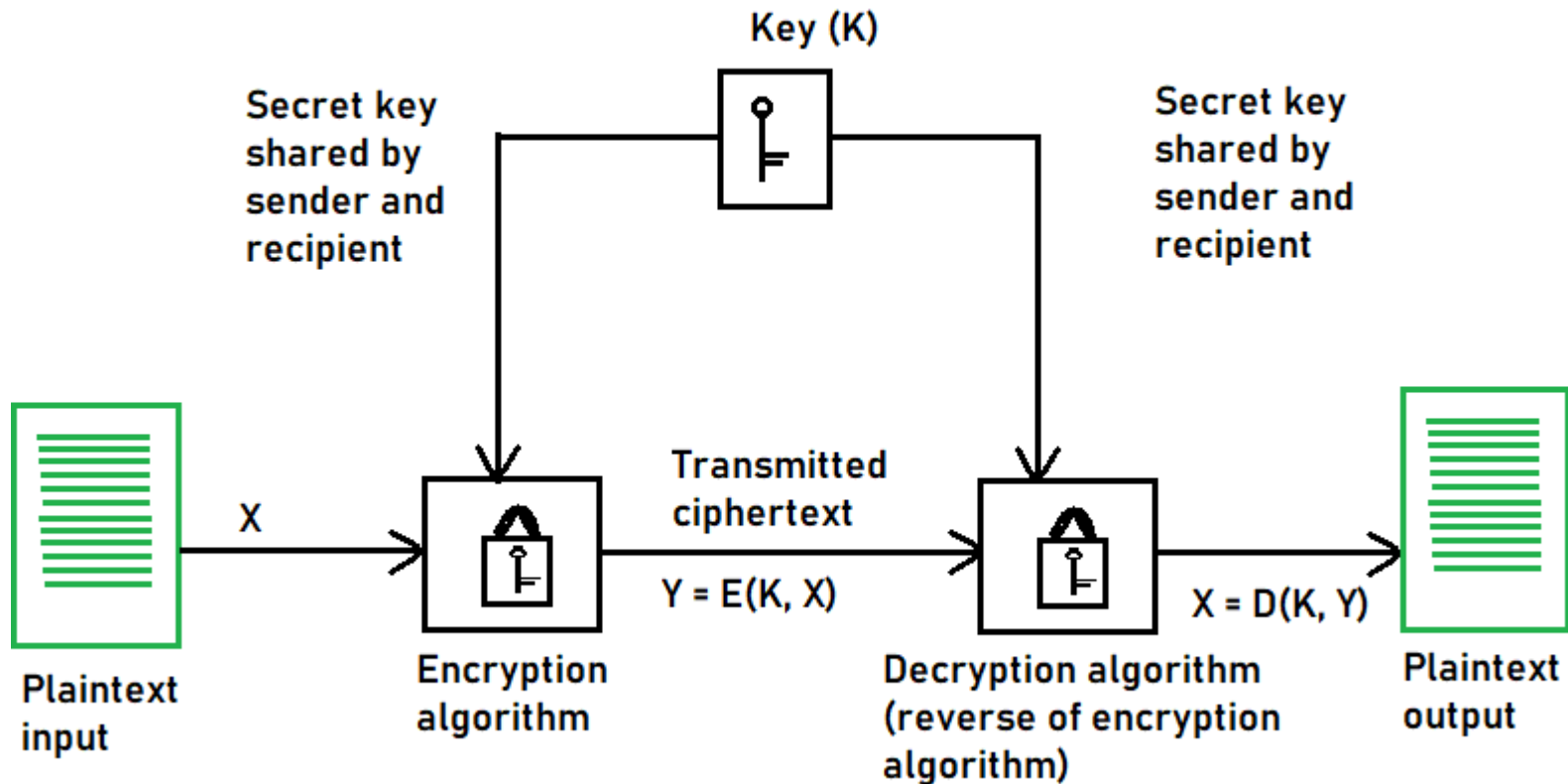
- **Key Generation:**
 - Caesar to choose the key of his cipher and the Sender/Receiver chose a shared secret key for the Vigenère cipher.
 - Modern cryptographic system key generation is usually automated.
- **Key Verification:**
 - Almost all cryptographic algorithms have some weak keys that should not be used (e.g., Caesar cipher ROT 0 or ROT 25).
 - With the help of key verification procedures, these keys can be regenerated if they occur.
- **Key Storage:**
 - Modern cryptographic system store keys in memory.
- **Key Exchange:**
 - Key management procedures should provide a secure key exchange mechanism over an untrusted medium.
- **Key Revocation and Destruction:**
 - Revocation notifies all interested parties that a certain key has been compromised and should no longer be used.
 - Destruction erases old keys in a manner that prevents malicious attackers from recovering them.

- Adding one bit to a key doubles the keyspace.
- For each bit added to the DES key, the attacker would require twice the amount of time to search the keyspace.
- Longer keys are more secure but are also more resource intensive and can affect throughput.



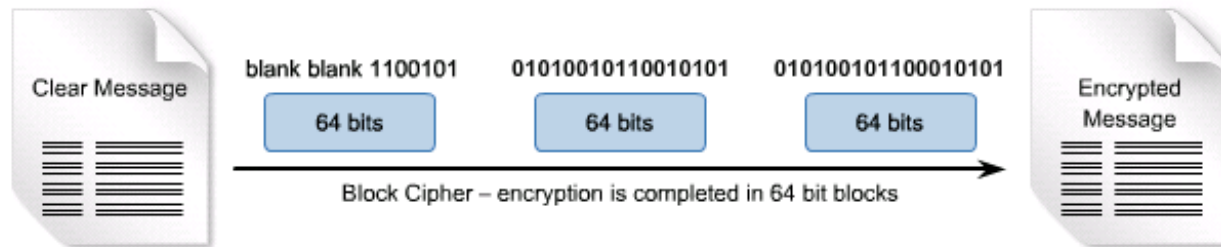
DES Key Length	Keyspace	# of Possible Keys
56 bit	2^{56}	72,000,000,000,000,000
57 bit	2^{57}	144,000,000,000,000,000
58 bit	2^{58}	288,000,000,000,000,000
59 bit	2^{59}	576,000,000,000,000,000

- Symmetric encryption algorithms, also called shared secret-key algorithms, **use the same pre-shared secret key to encrypt and decrypt data.**
 - The **pre-shared key is known by the sender and receiver** before any encrypted communications begins.
- Because both parties are guarding a shared secret, the encryption algorithms used can have shorter key lengths.
 - Shorter key lengths mean faster execution.
- For this reason symmetric algorithms are generally much less computationally intensive than asymmetric algorithms.

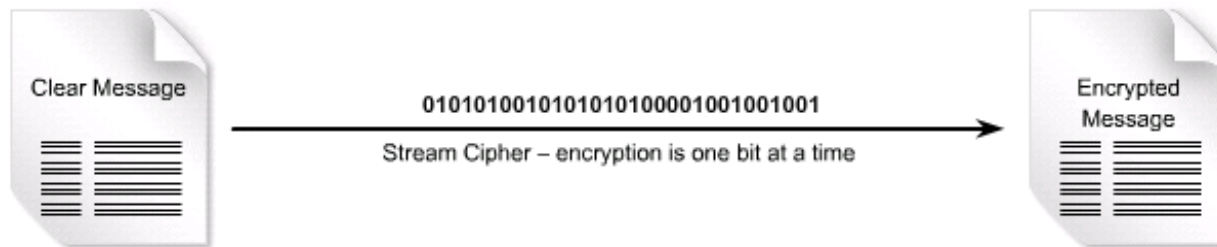


- <https://www.geeksforgeeks.org/symmetric-key-cryptography/>

- Block ciphers transform a fixed-length block of plaintext into a common block of ciphertext of 64 or 128 bits.
 - Block size refers to how much data is encrypted at any one time.



- Stream ciphers encrypt plaintext one byte or one bit at a time.
 - Think of it like a block cipher with a block size of one bit.
 - The Vigenère cipher is an example of a stream cipher.
 - Can be much faster than block ciphers, and generally do not increase the message size.



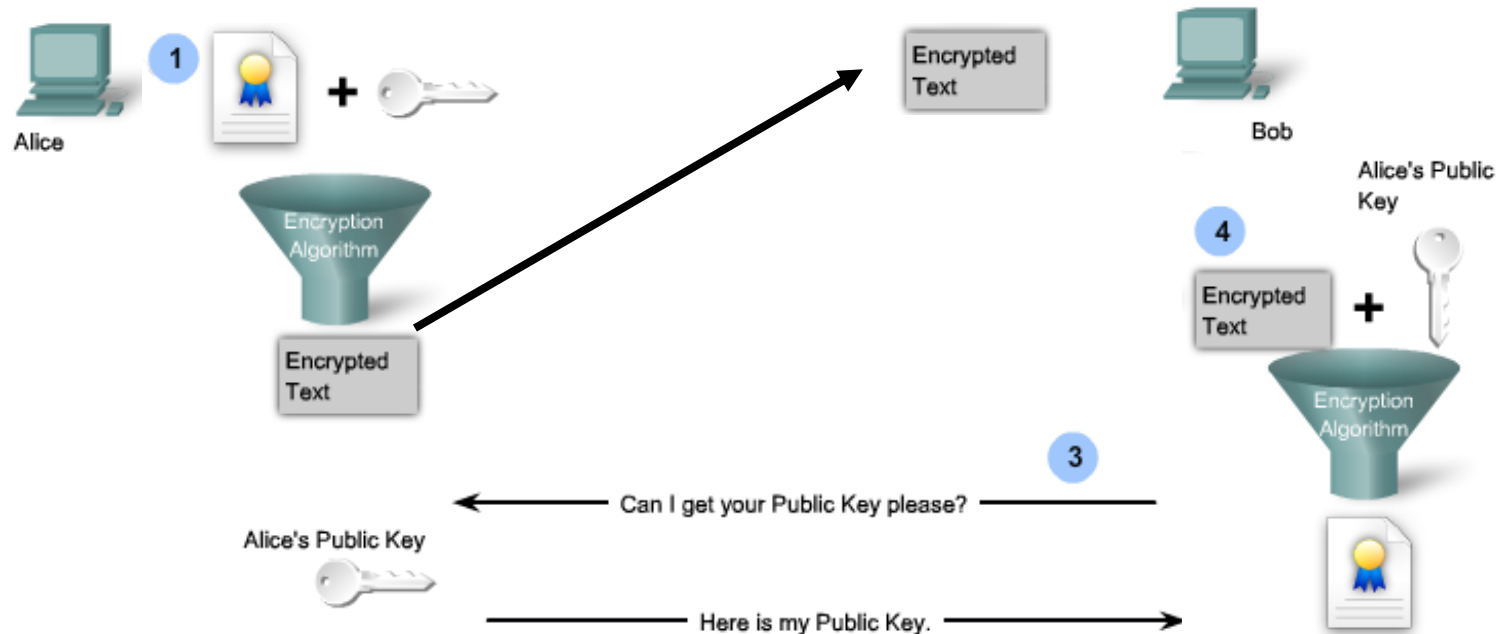
Algorithm	Description, Key Sizes, and Author
AES (Advanced Encryption Standard)	Developed by Vincent Rijmen and Joan Daemen. A widely-used symmetric encryption standard adopted by the U.S. government. Uses substitution-permutation network (SPN) and supports key sizes of 128, 192, and 256 bits. Commonly used in TLS/SSL, VPNs, and disk encryption.
DES (Data Encryption Standard)	Developed by IBM, with input from the NSA. A symmetric block cipher from the 1970s with a 56-bit key. Uses 16 rounds of Feistel structure but is now obsolete due to brute-force vulnerabilities.
3DES (Triple DES)	Developed by IBM. Improves DES by applying the algorithm three times to each data block. Uses two or three 56-bit keys (112 or 168 bits total). Common in legacy systems.
Blowfish	Developed by Bruce Schneier. A symmetric block cipher with a 64-bit block size and variable key sizes (32 to 448 bits). Known for speed and used in password hashing and file encryption.
Twofish	Developed by Bruce Schneier and his team. A successor to Blowfish with a 128-bit block size and keys up to 256 bits. Known for flexibility and used in TrueCrypt and other encryption software.
RC4 (Rivest Cipher 4)	Developed by Ron Rivest. A stream cipher known for simplicity and speed, using key sizes from 40 to 2048 bits. Once popular in WEP and SSL/TLS but now considered insecure.
ChaCha20	Developed by Daniel J. Bernstein. A modern stream cipher providing high security and performance. Uses a 256-bit key and is based on a variant of the Salsa20 algorithm. Commonly used in TLS 1.3, OpenSSH, and mobile VPNs.
IDEA (International Data Encryption Algorithm)	Developed by James Massey and Xuejia Lai. A symmetric block cipher known for use in PGP (Pretty Good Privacy). Uses a 64-bit block size and a 128-bit key. IDEA is secure but patented, limiting its widespread use.
GOST (Gosudarstvennyy Standard)	Developed by the Soviet Academy of Sciences. A symmetric cipher with a 256-bit key and 32 rounds of Feistel structure. Part of Russian national standards, used for secure communications.
Camellia	Developed by Mitsubishi Electric and NTT. A symmetric block cipher similar in design to AES. Supports key sizes of 128, 192, and 256 bits. Optimized for software and hardware implementation, commonly used in SSL/TLS and mobile security.

- Asymmetric encryption algorithms, also called public/private key algorithms, **use two different keys to encrypt and decrypt data**
- Secure messages can be exchanged without having to have a pre-shared key.
- Because both parties do not have a shared secret, very long key lengths must be used to thwart attackers.
 - These algorithms are resource intensive and slower to execute.
- *In practice, asymmetric algorithms are typically 100 to 1,000 times slower than symmetric algorithms.*



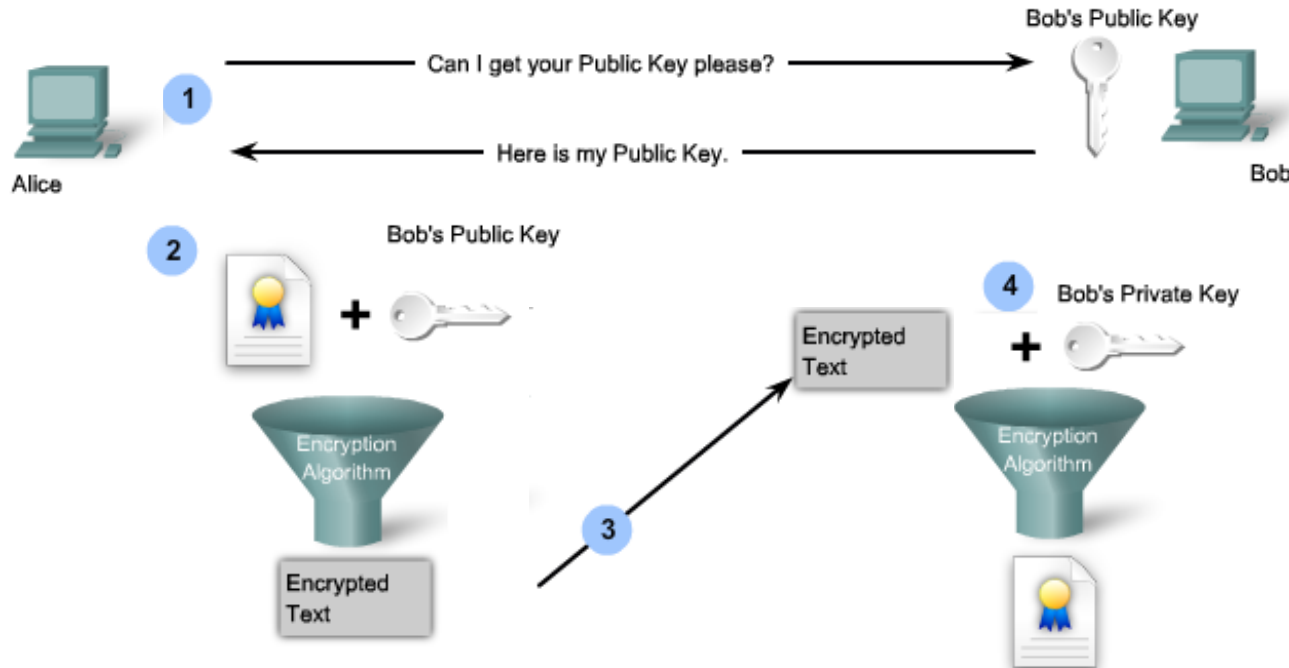
- <https://www.geeksforgeeks.org/asymmetric-key-cryptography/>

Private Key (Encrypt) + Public Key (Decrypt) = Authentication

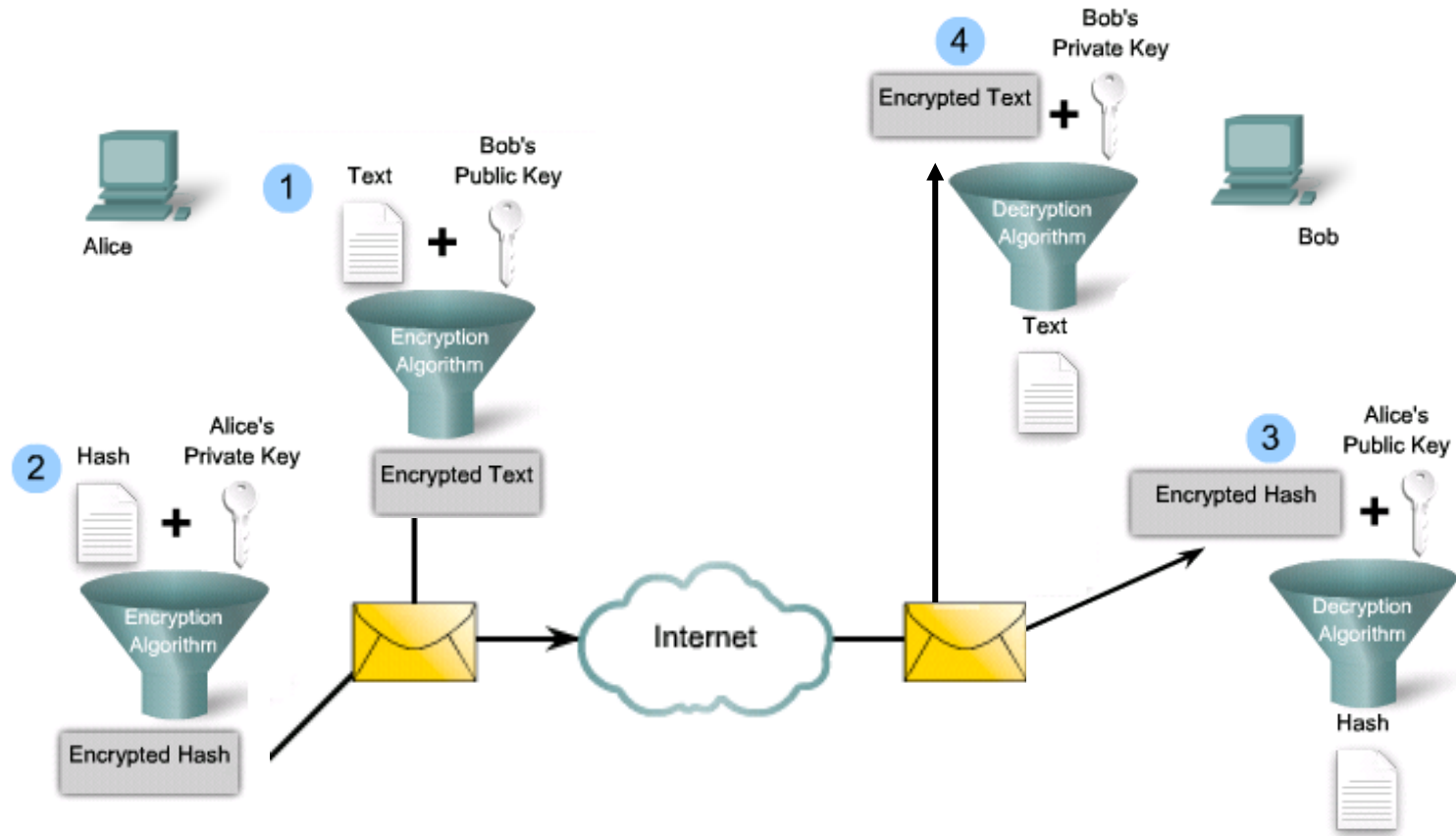


- 1) Alice will encrypt the message with her own private key.
- 2) Alice sends the ciphertext to Bob.
- 3) Bob requests Alice's public key to authenticate the message.
- 4) To verify that the message is from Alice, Bob uses Alice's public key to decrypt the message. If the message is readable, the authenticity of Alice's message is verified.

Public Key (Encrypt) + Private Key (Decrypt) = Confidentiality



- 1) Alice asks Bob for his public key.
- 2) Alice uses Bob's public key to encrypt the message using a pre-agreed algorithm.
- 3) Alice sends Bob the cipher-text.
- 4) Bob uses his private key to decrypt the message.

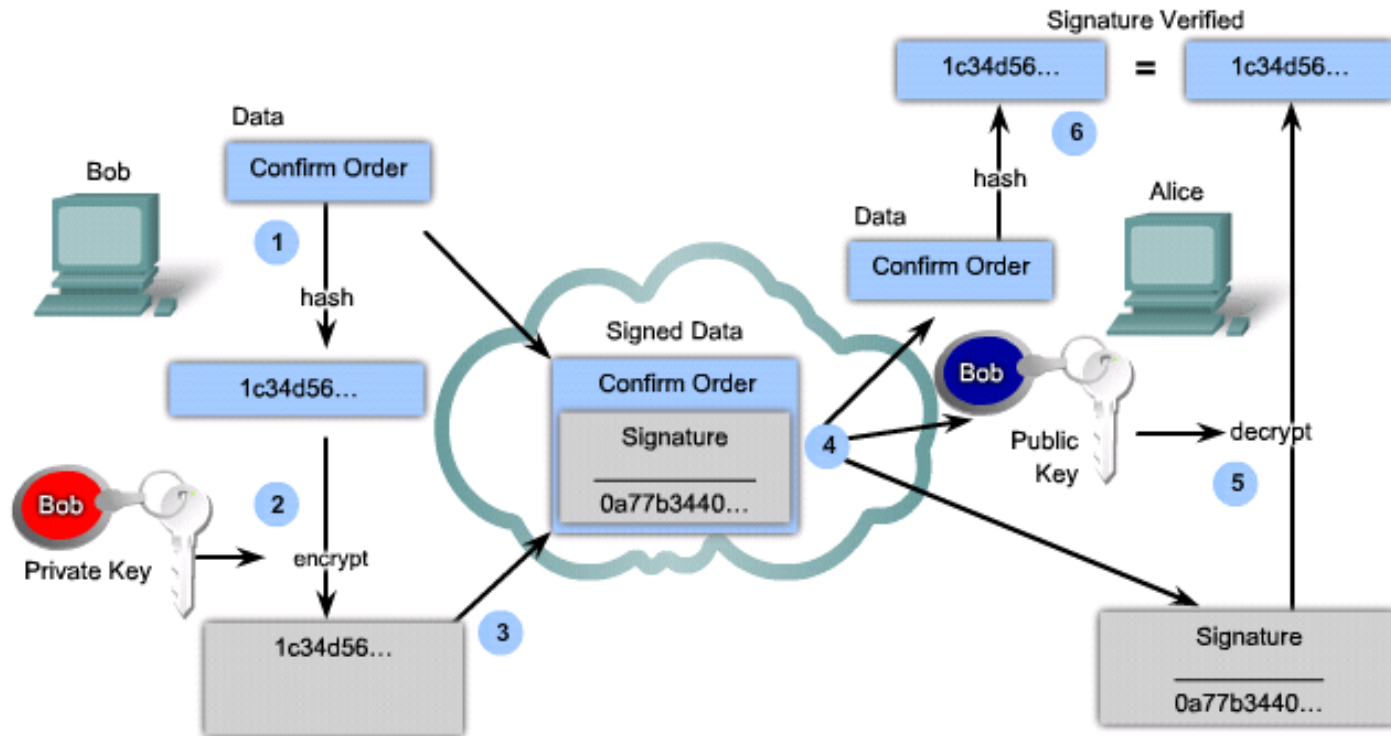


- 1) Alice encrypts the message using Bob's public key.
- 2) Alice encrypts the HMAC with her own private key.
- 3) Bob uses Alice's public key to authenticate the HMAC message.
- 4) Bob uses his private key to decrypt the cipher-text.

Algorithm	Description, Key Sizes, and Author
RSA (Rivest-Shamir-Adleman)	Developed by Ron Rivest, Adi Shamir, and Leonard Adleman. RSA is one of the most widely used public-key cryptosystems. It uses key sizes from 1024 to 4096 bits and is based on the difficulty of factoring large integers. Commonly used for digital signatures and secure data transmission.
Diffie-Hellman Key Exchange	Developed by Whitfield Diffie and Martin Hellman. Diffie-Hellman is a method for securely exchanging cryptographic keys over a public channel. It is the basis for many secure protocols but does not provide encryption or digital signatures alone.
ECC (Elliptic Curve Cryptography)	Developed by Neal Koblitz and Victor S. Miller. ECC provides the same level of security as RSA with smaller key sizes (256-bit ECC is equivalent to 3072-bit RSA). Widely used in modern cryptographic systems, including TLS, Bitcoin, and mobile security.
ElGamal	Developed by Taher Elgamal. A public-key cryptosystem based on the Diffie-Hellman key exchange. Commonly used in digital signature schemes and hybrid encryption protocols. Uses key sizes of 1024 bits and above for security.
DSA (Digital Signature Algorithm)	Developed by the U.S. National Security Agency (NSA). DSA is a Federal Information Processing Standard (FIPS) for digital signatures. It is based on modular exponentiation and is commonly used for authentication and integrity verification. Key sizes range from 1024 to 3072 bits.
Ed25519	Developed by Daniel J. Bernstein. Ed25519 is a high-performance, secure elliptic curve algorithm used for digital signatures. It is based on the Edwards curve and provides strong security with a compact 256-bit key size. Popular in modern security protocols such as OpenSSH and TLS.
Paillier Cryptosystem	Developed by Pascal Paillier. A probabilistic asymmetric algorithm known for its homomorphic properties. Paillier is used in privacy-preserving computations and secure voting systems. Typical key sizes range from 1024 to 4096 bits.
McEliece Cryptosystem	Developed by Robert McEliece. A code-based public-key cryptosystem known for its resistance to quantum attacks. It uses large key sizes (typically 512 KB to 1 MB) but is highly secure against current and post-quantum attacks.
NTRUEncrypt	Developed by Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. NTRU is a lattice-based asymmetric encryption algorithm designed for high security and efficiency. It is considered a strong candidate for post-quantum cryptography. Key sizes vary but are generally compact compared to other lattice-based systems.

PKI

- Digital signatures are often used in the following situations:
 - To provide a unique proof of data source, which can only be generated by a single party, such as contract signing in e-commerce environments.
 - To authenticate a user by using the private key of that user and the signature it generates.
 - To prove the authenticity and integrity of PKI certificates.
 - To provide nonrepudiation using a secure timestamp and a trusted time source.
 - Each party has a unique, secret signature key, which is not shared with any other party, making nonrepudiation possible.
- Authenticity of digitally signed data:
 - Digital signatures authenticate a source, proving that a certain party has seen and signed the data in question.
- Integrity of digitally signed data:
 - Digital signatures guarantee that the data has not changed from the time it was signed.
- Nonrepudiation of the transaction:
 - The recipient can take the data to a third party, and the third party accepts the digital signature as a proof that this data exchange did take place.
 - The signing party cannot repudiate that it has signed the data.

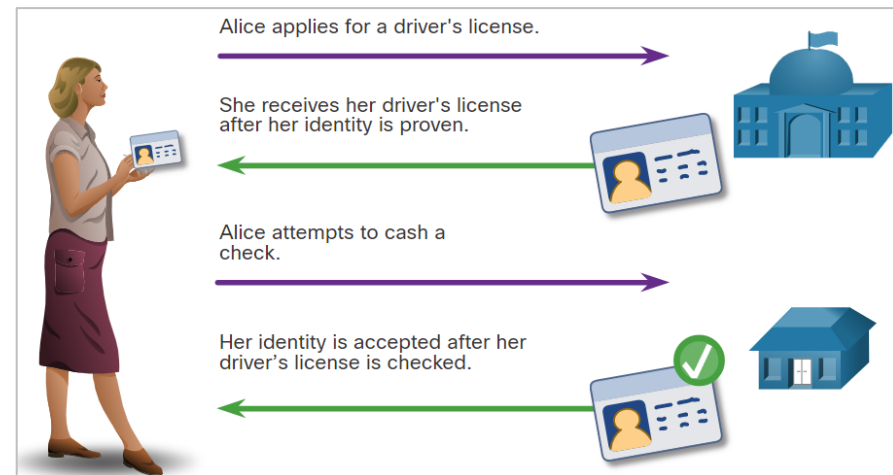


- 1) Bob creates a hash of the document.
- 2) Bob encrypts the hash with the private key.
- 3) The encrypted hash, known as the signature, is appended to the document.
- 4) Alice accepts the document with the digital signature and obtains Bob's public key
- 5) Alice decrypts the signature using Bob's public key to unveil the assumed hash value
- 6) Alice calculates the hash of the received document, without its signature, and compares this hash to the decrypted signature hash and if the hashes match = document is authentic.

Algorithm	Description, Author, and Key Sizes
DSA (Digital Signature Algorithm)	Developed by the NSA and adopted by NIST as part of the Digital Signature Standard (DSS). Uses modular exponentiation and discrete logarithms. Key sizes: 1024 to 3072 bits. Commonly used in government and secure communication protocols.
RSA (Rivest–Shamir–Adleman)	Developed by Ron Rivest, Adi Shamir, and Leonard Adleman. Based on the difficulty of factoring large integers. Supports both encryption and digital signatures. Key sizes: 1024 to 4096 bits. Widely used in SSL/TLS and digital certificates.
ECDSA (Elliptic Curve Digital Signature Algorithm)	Developed by Neal Koblitz and Victor S. Miller. An elliptic curve variant of DSA, providing the same security with smaller key sizes. Key sizes: 160 to 521 bits. Commonly used in modern cryptography, including Bitcoin and TLS.
Ed25519	Developed by Daniel J. Bernstein. A high-performance elliptic curve digital signature algorithm. Known for speed and security. Uses a 256-bit key (Ed25519 curve). Popular in OpenSSH and modern cryptographic protocols.
Schnorr Signature	Developed by Claus Schnorr. A digital signature scheme known for simplicity and efficiency. Recently adopted in Bitcoin (Taproot upgrade). Key sizes: 256 bits (commonly with elliptic curves).
GOST R 34.10	Developed by the Russian Federal Agency for Technical Regulation and Metrology. A digital signature algorithm based on elliptic curve cryptography. Key sizes: 256 bits. Used in Russian government standards for secure communication.
Falcon	Developed by Pierre-Alain Fouque, Thomas Prest, and others. A lattice-based post-quantum digital signature algorithm. Designed for security against quantum computers. Key sizes: 512 to 1024 bits. A finalist in the NIST post-quantum cryptography competition.
Rainbow	Developed by Jintai Ding and Dieter Schmidt. A multivariate polynomial-based digital signature algorithm. Post-quantum secure. Key sizes: Varies depending on security level, typically large due to multivariate nature.

- When establishing an asymmetric connection between two hosts, the hosts will exchange their public key information.
 - Trusted third parties on the Internet validate the authenticity of these public keys using digital certificates. The third-party issues credentials that are difficult to forge.
 - From that point forward, all individuals who trust the third party simply accept the credentials that the third-party issues.
- **The Public Key Infrastructure (PKI) consists of specifications, systems, and tools that are used to create, manage, distribute, use, store, and revoke digital certificates.**
 - The Certificate Authority (CA) creates digital certificates by tying a public key to a confirmed identity, such as a website or individual.

- Illustrates how a driver's license is analogous to a digital certificate

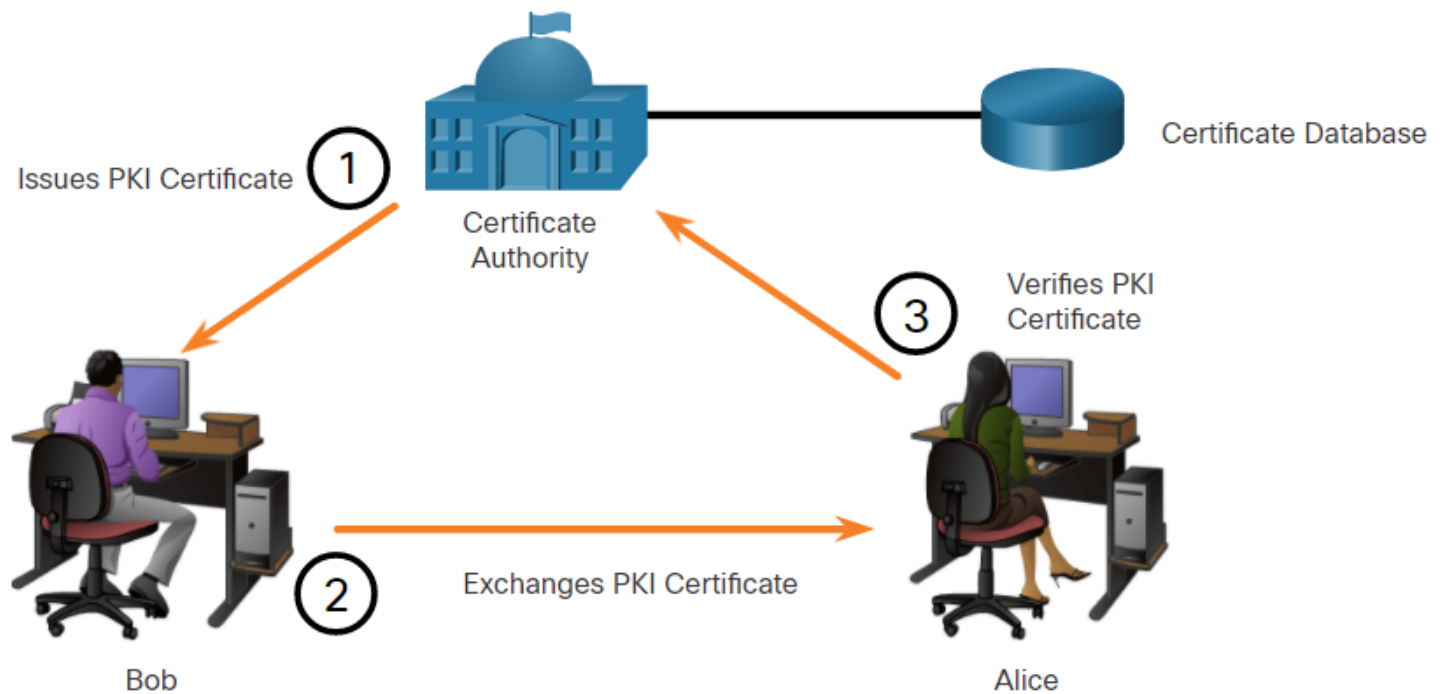


- PKI is needed to support large-scale distribution and identification of public encryption keys.



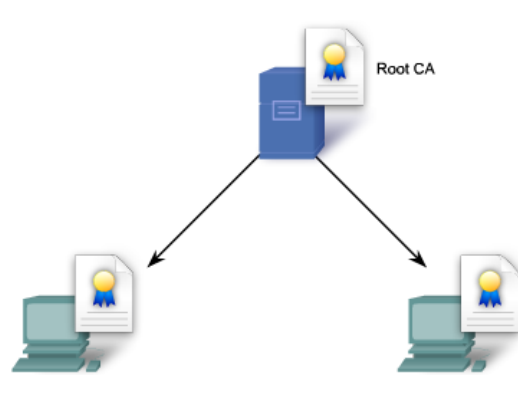
1. PKI certificates contain an entity's or individual's public key, its purpose, the certificate authority (CA) that validated and issued the certificate, the date range during which the certificate is valid, and the algorithm used to create the signature.
2. The certificate store resides on a local computer and stores issued certificates and private keys.
3. The PKI Certificate of Authority (CA) is a trusted third party that issues PKI certificates to entities and individuals after verifying their identity. It signs these certificates using its private key.
4. The certificate database stores all certificates approved by the CA.

Component	Detailed Description
Root Certificate Authority (Root CA)	The highest level of trust in a PKI hierarchy. A Root CA is a trusted authority that issues certificates to subordinate CAs, creating a chain of trust. The Root CA's public key is distributed and stored in web browsers, operating systems, and devices by default. If the Root CA is compromised, the entire trust chain is broken. Examples: DigiCert, GlobalSign, Entrust, Let's Encrypt, and Sectigo.
Certificate Authority (CA)	A trusted entity responsible for issuing, managing, and revoking digital certificates. The CA verifies the identity of applicants and digitally signs their certificates using its private key. CAs can be classified into Root CAs and Intermediate CAs.
Intermediate Certificate Authority (Intermediate CA)	Acts as a bridge between the Root CA and end-entity certificates. It is authorized by a Root CA and can issue certificates to organizations or individuals. Intermediate CAs help compartmentalize risk; if compromised, only its issued certificates are affected.
Registration Authority (RA)	An RA acts as an intermediary between the certificate applicant and the CA. It validates the applicant's identity and verifies the information provided in the Certificate Signing Request (CSR). Once verified, it forwards the request to the CA for issuance. RAs are commonly used by large organizations for internal certificate management.
Digital Certificate	An electronic document binding a public key to the identity of an individual, organization, or device. It includes: Public Key, Certificate Holder's Identity, CA's Digital Signature, Certificate Validity Period, and Serial Number. Digital certificates are used for SSL/TLS, email encryption, document signing, and code signing.
Public and Private Keys	The key pair at the core of asymmetric cryptography: • Public Key: Shared openly, used for encryption or signature verification. • Private Key: Kept secret, used for decryption or signing. Public keys are distributed via digital certificates, and private keys are stored securely (e.g., in Hardware Security Modules (HSMs)).
Certificate Revocation List (CRL)	A list published by the CA containing digital certificates that have been revoked before their expiration date. Reasons for revocation: Private Key Compromise, Certificate Misuse, or Organizational Changes. Users check the CRL to ensure a certificate is valid before secure communication.
Online Certificate Status Protocol (OCSP)	A real-time protocol for checking the status of a digital certificate without downloading the entire CRL. Clients send an OCSP request to the CA's OCSP responder, which returns the certificate status (Good, Revoked, or Unknown). OCSP is faster and more efficient than CRLs, making it suitable for web browsers and real-time applications.

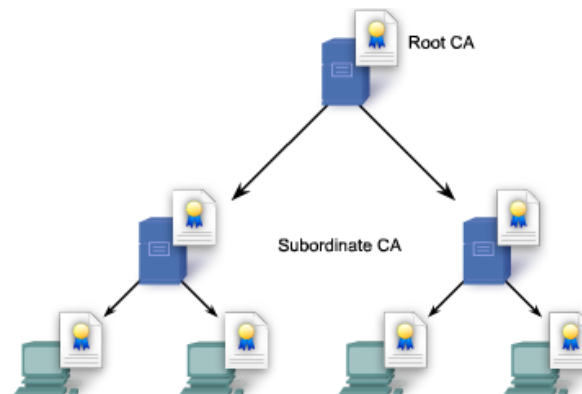


- 1. Issues PKI Certificate.** Bob initially requests a certificate from the CA. The CA authenticates Bob and stores Bob's PKI certificate in the certificate database.
- 2. Exchanges PKI Certificate.** Bob communicates with Alice using his PKI certificate.
- 3. Verifies PKI Certificate.** Alice communicates with the trusted CA using the CA's public key. The CA refers to the certificate database to validate Bob's PKI certificate.

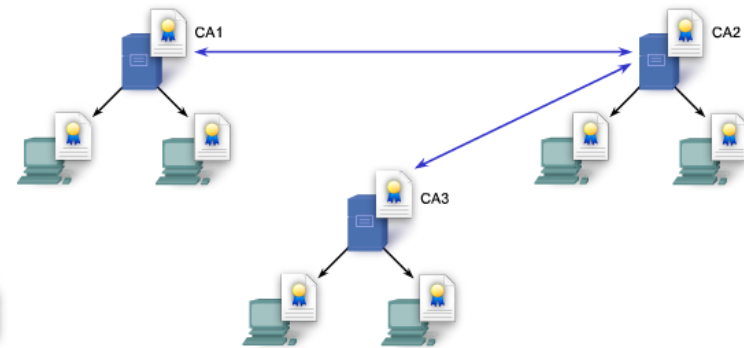
- PKIs can form different **topologies / levels of trust** which are as follows
- Single-Root PKI Topology:** The simplest is the single-root PKI topology. The root CA issues all the certificates to the end users within the same organization. On larger networks, PKI CAs may be linked using two basic architectures:
 - Hierarchical CA topologies:** The root CA (highest level CA), can issue certificates to end users and to a subordinate CA.
 - Cross-certified CA topologies:** A peer-to-peer model in which individual CAs establish trust relationships with other CAs by cross-certifying CA certificates.



Single-Root PKI Topology



Hierarchical CA Topologies

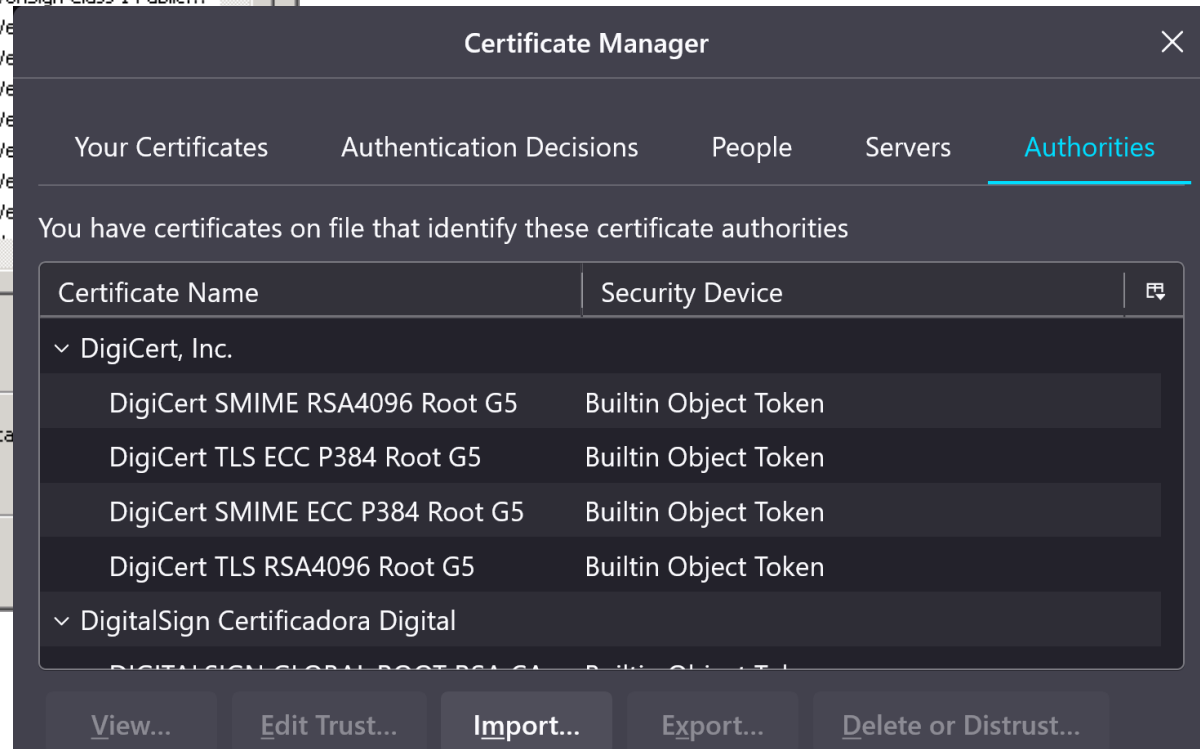
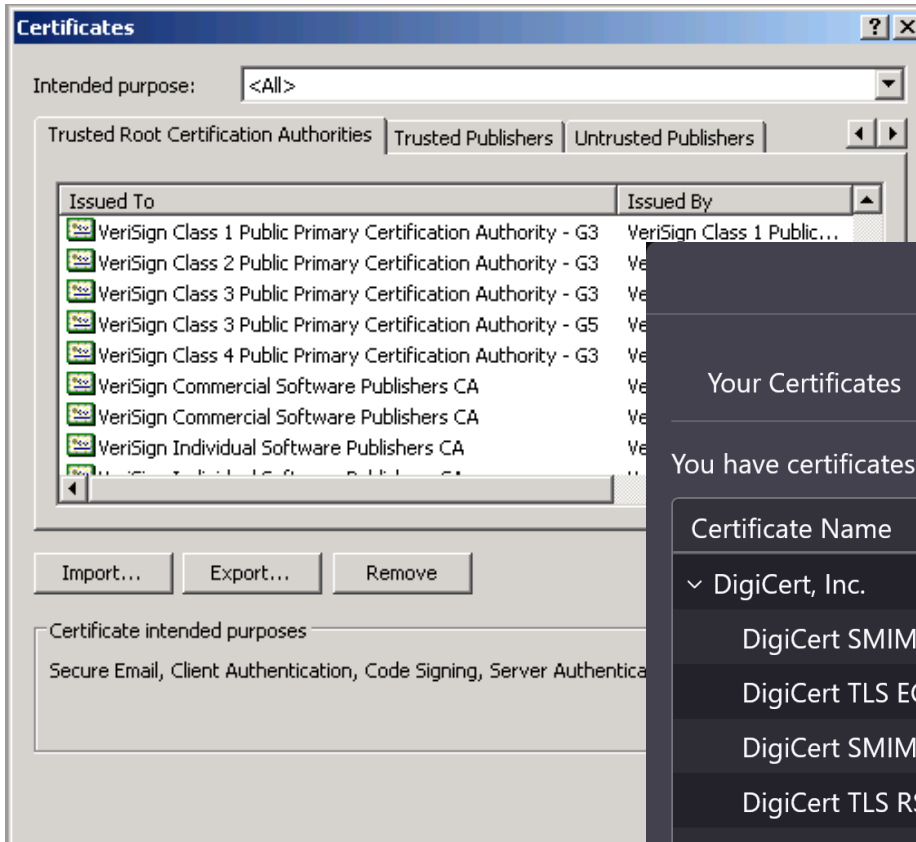


Cross-certified CA Topologies

- Many vendors provide CA servers as a managed service or as an end-user product.
 - Organizations may also implement private PKIs using Microsoft Server or Open SSL.
- CAs issue certificates based on classes which determine how trusted a certificate is.
 - The class number is determined by how rigorous the procedure was that verified the identity of the holder when the certificate was issued.
 - The higher the class number, the more trusted the certificate.

Class	Description
0	Used for testing in situations in which no checks have been performed.
1	Used by individuals who require verification of email.
2	Used by organizations for which proof of identity is required.
3	Used for servers and software signing.
4	Used for online business transactions between companies.
5	Used for private organizations or government security.

- Some CA public keys are preloaded, such as those listed in web browsers.





<http://www.verisign.com>



<http://www.entrust.com>



<http://www.verizonbusiness.com/>



<http://www.novell.com>



<http://www.microsoft.com>



Domain Validation (DV) SSL Certificate

EASY TO INSTALL SECURITY

Best for verifying your website with Google, and helping boost Google rankings. You get SHA-2 and 2048-bit encryption and a trust indicator in the address bar.

[Learn more about SSL Certificates](#)

AS LOW AS

€54.99 /yr

49% savings with a 3-yr term

See Plans



Managed SSL Service

MANAGED FOR YOU SECURITY

Best for getting an SSL fully installed and managed by us that verifies your site and helps boost Google rankings. Includes SHA-2 and 2048-bit encryption and a trust indicator in the address bar.

AS LOW AS

€84.99 /yr

50% savings with a 2-yr term

See Plans



Website Security

EXTENSIVE SECURITY

Best for helping to secure your site with a firewall, malware scanning, SSL certificate, annual site cleanup, and Blocklist monitoring.

AS LOW AS

€5.49 /mo

50% savings with a 2-yr term

See Plans

Basic - OV

Complete business-level validation, reinforcing customer confidence in online presence while securing customer data

☐ Convert this to a wildcard domain

\$26 / month / standard domain

12 month auto-renewing subscription \$312.00

ADD TO CART

[LEARN MORE >](#)

Includes:

- ✓ Unlimited certificate issuance/replacement
- ✓ 24x5 standard support
- ✓ DigiCert CertCentral management portal

Secure Site - OV

Business-level validation with priority validation, management tools and faster issuance

☐ Convert this to a wildcard domain

\$44 / month / standard domain

12 month auto-renewing subscription \$528.00

ADD TO CART

[LEARN MORE >](#)

Includes:

- ✓ Basic - OV benefits, plus
- ✓ Priority order processing
- ✓ DigiCert Smart Seal
- ✓ Domain reputation monitoring
- ✓ Vulnerability assessment scans

RECOMMENDED

Secure Site Pro - OV

Business-level validation, priority validation, and enhanced security for domain and brand protection

☐ Convert this to a wildcard domain

\$108 / month / standard domain

12 month auto-renewing subscription \$1,296.00

ADD TO CART

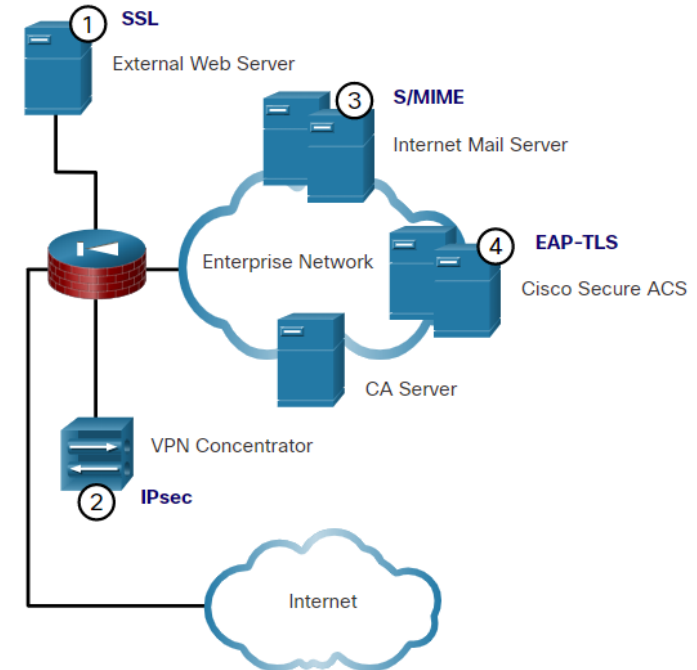
[LEARN MORE >](#)

Includes:

- ✓ Basic - OV benefits, plus
- ✓ Priority order processing
- ✓ DigiCert Smart Seal
- ✓ Domain reputation monitoring
- ✓ Vulnerability assessment scans
- ✓ Brand protection (CT log monitoring)

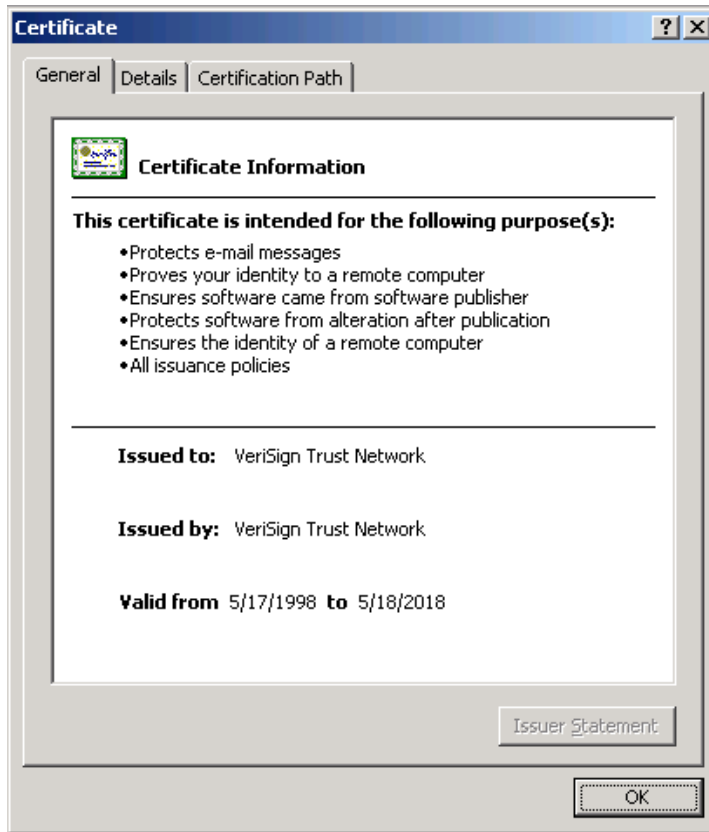
- Interoperability between a PKI and its supporting services is a concern because many CA vendors have proposed and implemented proprietary solutions.
- To address this interoperability concern, the IETF published the Internet X.509 Public Key Infrastructure Certificate Policy and Certification Practices Framework (RFC 2527).
- **The X.509 version 3 (X.509 v3) standard defines the format of a digital certificate.**

- X.509v3 Applications



1. **SSL** - Secure web servers use X.509.v3 for website authentication in the SSL and TLS protocols, while web browsers use X.509v3 to implement HTTPS client certificates. SSL is the most widely used certificate-based authentication.
2. **IPsec** - IPsec VPNs use X.509 when certificates can be used as a public key distribution mechanism for internet key exchange (IKE) RSA-based authentication.
3. **S/MIME** - User mail agents that support mail protection with the Secure/Multipurpose Internet Mail Extensions (S/MIME) protocol use X.509 certificates.
4. **EAP-TLS** - Cisco switches can use certificates to authenticate end devices that connect to LAN ports using 802.1.x between the adjacent devices. The authentication can be proxied to a central ACS via the Extensible Authentication Protocol with TLS (EAP-TLS).

- Defines basic PKI formats such as the certificate and certificate revocation list (CRL) format to enable basic interoperability



PostSignum Root QCA 4	
Subject Name	
Country	CZ
Organization	NTRCZ-47114983
Common Name	Česká pošta, s.p.
	PostSignum Root QCA 4
Issuer Name	
Country	CZ
Organization	NTRCZ-47114983
Common Name	Česká pošta, s.p.
	PostSignum Root QCA 4
Validity	
Not Before	Thu, 26 Jul 2018 09:56:08 GMT
Not After	Mon, 26 Jul 2038 09:56:08 GMT
Public Key Info	
Algorithm	RSA
Key Size	4096
Exponent	65537
Modulus	C6:66:8D:82:A0:7E:BE:8B:22:25:78:10:C0:08:8B:9A:19:7F:D5:AD:00:14:0E:64:DE:D...
Miscellaneous	
Serial Number	0F:A0
Signature Algorithm	SHA-512 with RSA Encryption
Version	3
Download	PEM (cert) PEM (chain)
Fingerprints	
SHA-256	AC:7F:78:62:E6:85:C7:A7:D9:82:6A:58:EA:32:D1:83:D4:89:3F:CC:8F:8F:D6:D9:00:C...
SHA-1	AA:40:D2:57:9B:A8:24:24:CD:27:71:9B:1D:6B:1F:35:71:73:80:99

Field	Description
Version	Specifies the version of the X.509 standard being used (e.g., V1, V2, V3).
Serial Number	A unique identifier assigned to the certificate by the Certificate Authority (CA).
Signature Algorithm	The algorithm used by the CA to sign the certificate (e.g., SHA-256 with RSA).
Issuer	The name of the Certificate Authority (CA) that issued the certificate.
Validity Period	The time frame during which the certificate is valid. Includes: • Not Before: The start date and time when the certificate becomes valid. • Not After: The expiration date and time when the certificate expires.
Subject	The entity that the certificate identifies (e.g., domain name or organization).
Subject Public Key Info	Information about the public key associated with the subject, including: • Public Key Algorithm: The algorithm (e.g., RSA, ECC). • Public Key Value: The actual public key.
Signature Value	The digital signature of the certificate, created by the CA using its private key.
Key Usage (V3 Extension)	Specifies the purposes for which the key may be used (e.g., digital signatures, key encipherment).
Extended Key Usage (V3 Extension)	Additional purposes for the certificate, such as client authentication or code signing.
Basic Constraints (V3 Extension)	Indicates if the certificate is a CA certificate and specifies the path length for certificate chaining.
Subject Alternative Name (SAN)	Lists additional identities for the certificate, such as multiple domain names or IP addresses (common for multi-domain SSL certificates).
Authority Key Identifier (V3 Extension)	Provides a unique identifier for the CA that issued the certificate, linking it to the issuer's public key.
CRL Distribution Points (V3 Extension)	Specifies where to find the CA's Certificate Revocation List (CRL) to check if the certificate has been revoked.
Authority Information Access (AIA)	Provides information about how to access the CA's OCSP responder for real-time certificate status checking.

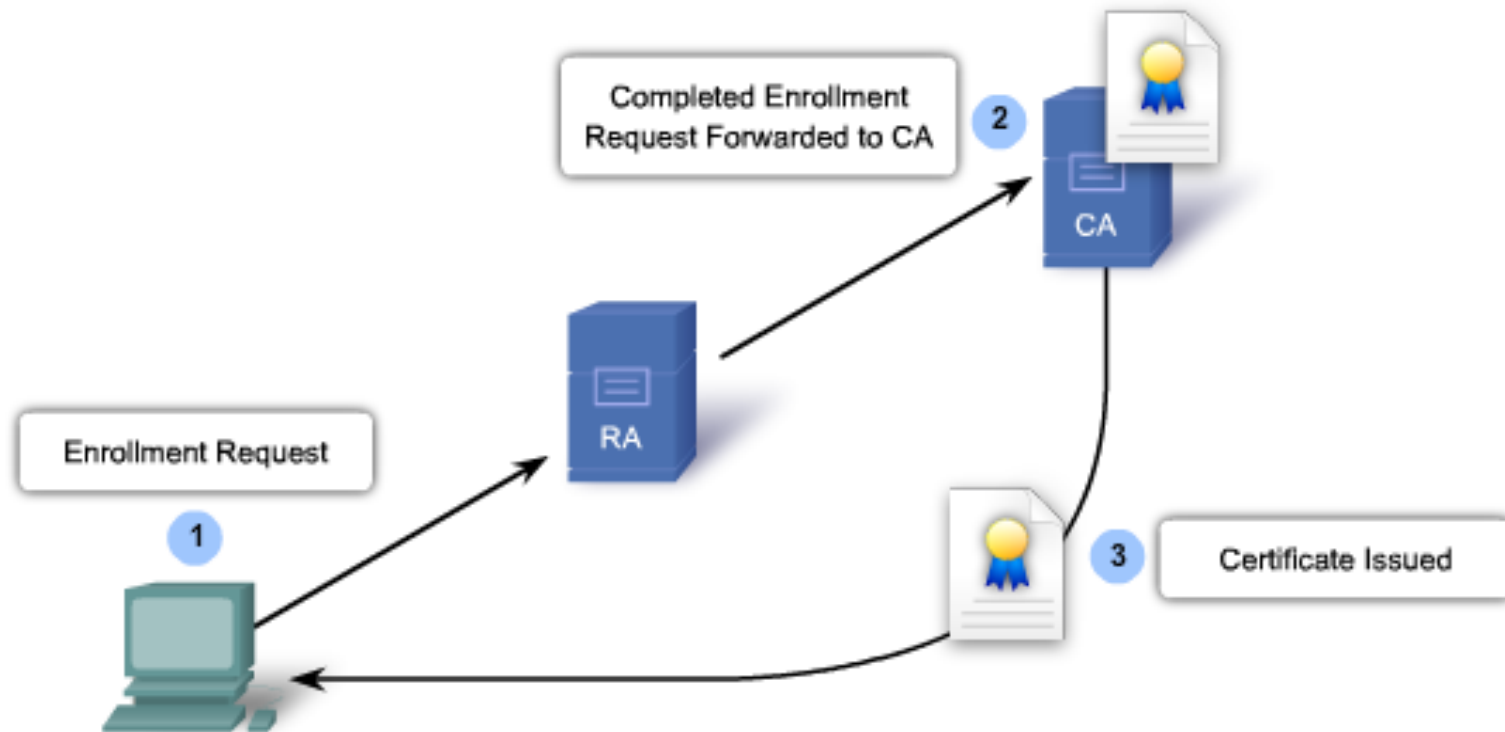
- The Public-Key Cryptography Standards (PKCS) refers to a group of Public Key Cryptography Standards devised and published by RSA Laboratories.
 - PKCS provides basic interoperability of applications that use public-key cryptography.
 - PKCS defines the low-level formats for the secure exchange of arbitrary data, such as an encrypted piece of data or a signed piece of data.

PKCS Standard	Title	Description
PKCS #1	RSA Cryptography Standard	Defines RSA encryption and signatures, including OAEP for encryption and PSS for signatures.
PKCS #3	Diffie-Hellman Key Exchange	Standard for securely establishing shared keys using the Diffie-Hellman protocol.
PKCS #5	Password-Based Encryption (PBE)	Describes methods for encrypting private keys using passwords (e.g., PBKDF2 for key derivation).
PKCS #7	Cryptographic Message Syntax (CMS)	Standard for digitally signed and encrypted messages, commonly used in S/MIME for secure email.
PKCS #8	Private Key Information Syntax	Defines the format for storing private keys (encrypted or unencrypted). Common in .p8 files.
PKCS #9	Attribute Types for PKCS	Defines standard attributes (e.g., email, challenge password) used in certificate requests (CSR).
PKCS #10	Certificate Signing Request (CSR)	Standard format for requesting a digital certificate from a Certificate Authority (CA).
PKCS #11	Cryptographic Token Interface	Standard for accessing cryptographic services on hardware security devices (e.g., HSMs, smart cards).
PKCS #12	Personal Information Exchange (PFX)	Format for storing certificates and private keys together (.pfx or .p12 files).
PKCS #13	Elliptic Curve Cryptography (ECC)	Describes standards for using ECC for strong security with shorter key lengths.
PKCS #15	Cryptographic Token Information Format	Defines the structure for storing cryptographic information on smart cards, including private keys and certificates.

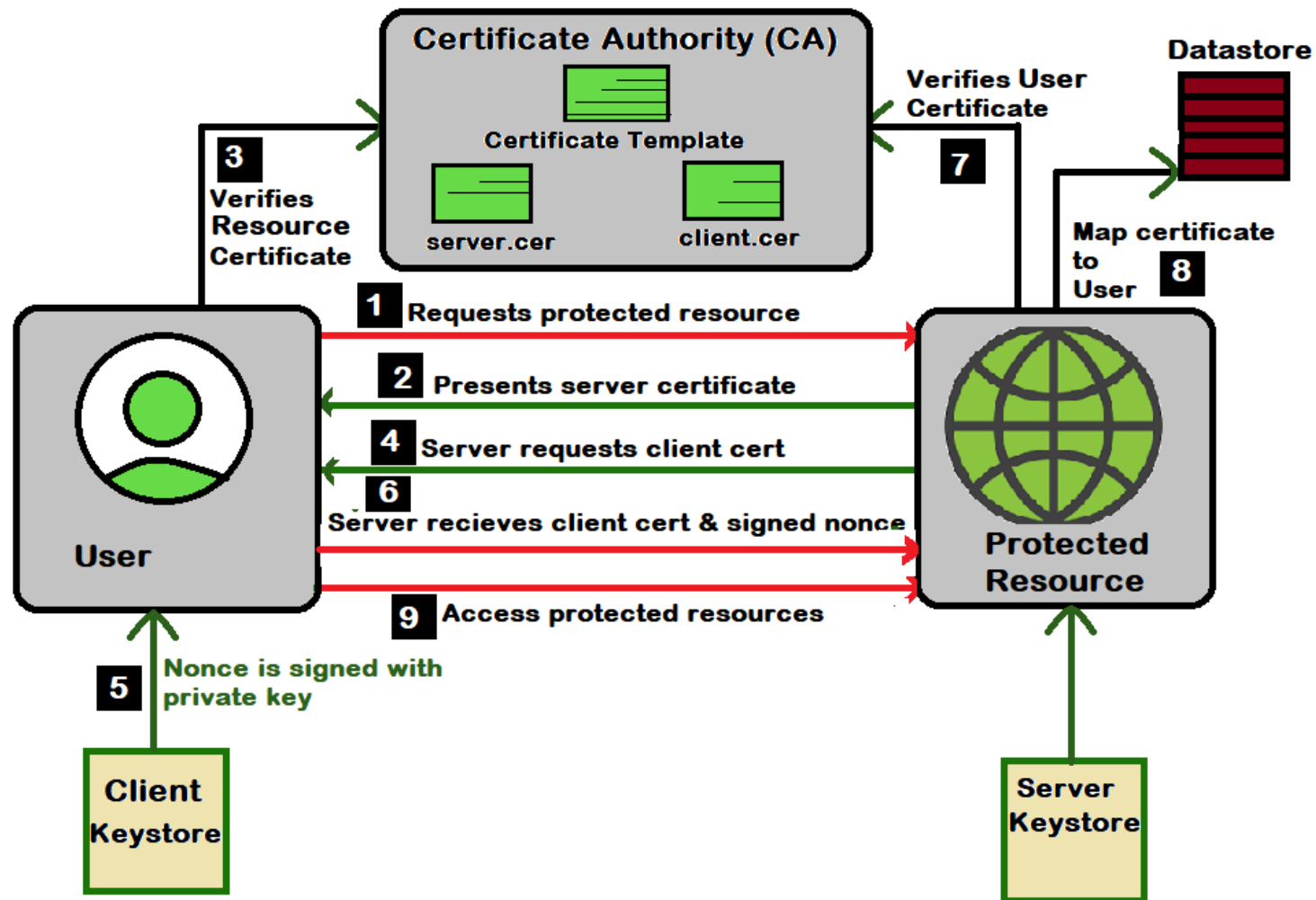
- Covers cryptographic standards beyond just certificates (keys, tokens, encryption).

File Type	Extension	Standard	Description
Certificate Signing Request (CSR)	.csr	PKCS #10	Used to request a digital certificate from a CA.
Private Key File	.p8, .key	PKCS #8	Stores private keys, optionally encrypted.
Personal Information Exchange	.pfx, .p12	PKCS #12	Stores private keys and certificates together.
Cryptographic Message Syntax	.p7b, .p7c	PKCS #7	Used for signed or encrypted messages.
Hardware Security Module Interface	N/A	PKCS #11	Interface standard for HSMs and smart cards.

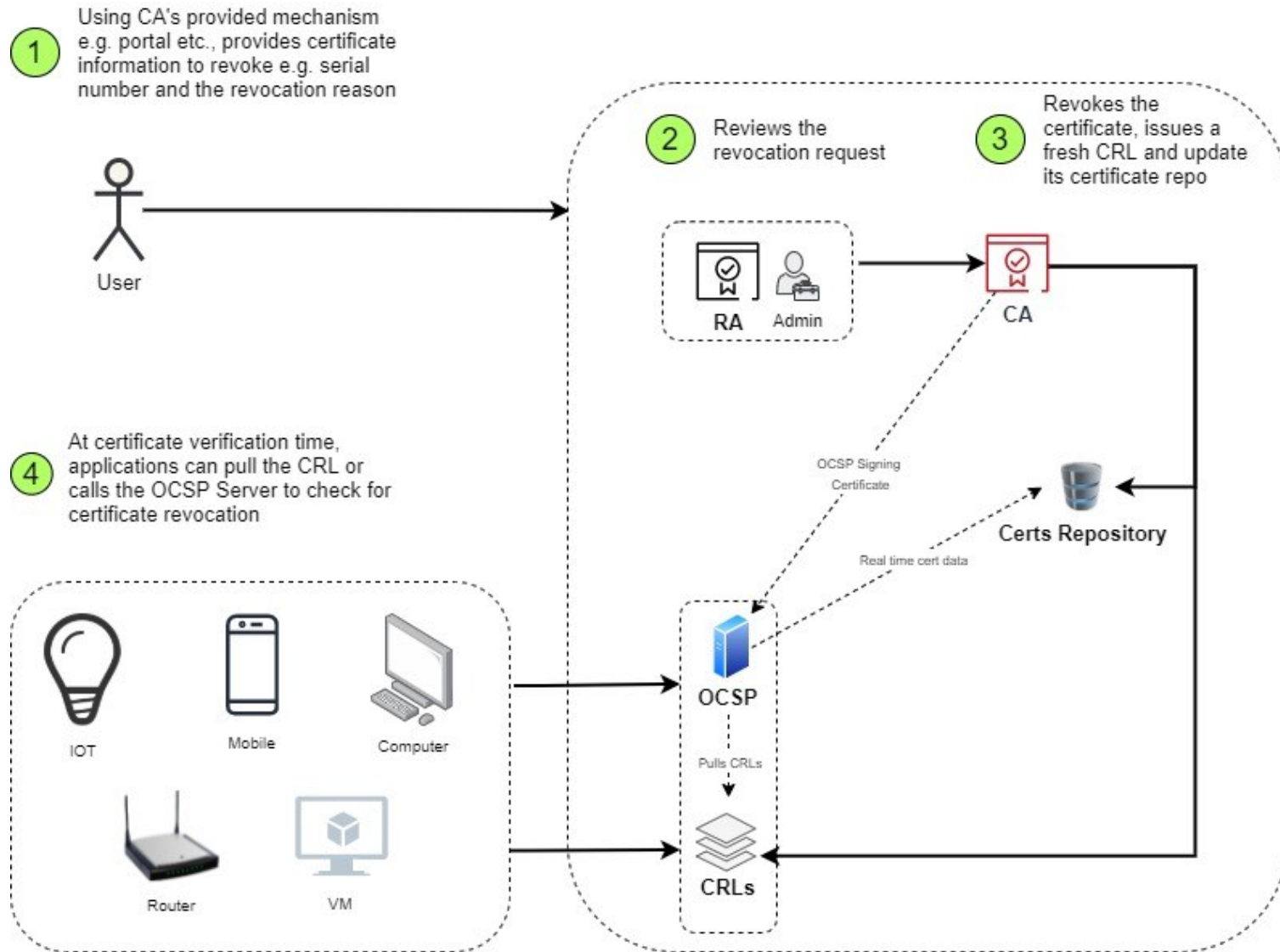
- All systems that leverage the PKI must have the CA's public key, which is called the self-signed certificate.
 - The CA public key verifies all the certificates issued by the CA and is vital for the proper operation of the PKI.
- The certificate enrollment process is used by a host system to enroll with a PKI. To do so, CA certificates are retrieved in-band over a network, and the authentication is done out-of-band (OOB) using the telephone.
 - The system enrolling with the PKI contacts a CA to request and obtain a digital identity certificate for itself and to get the CA's self-signed certificate.
 - The final stage verifies that the CA certificate was authentic and is performed using an out-of-band method such as the POTS to obtain the fingerprint of the valid CA identity certificate.
- A digital certificate can be revoked if key is compromised or if it is no longer needed.
- Only a root CA can issue a self-signed certificate that is recognized or verified by other CAs within the PKI.



1. Hosts will submit certificate requests to the RA.
2. After the Registration Authority adds specific information to the certificate request and the request is approved under the organization's policy, it is forwarded on to the Certification Authority.
3. The CA will sign the certificate request and send it back to the host.

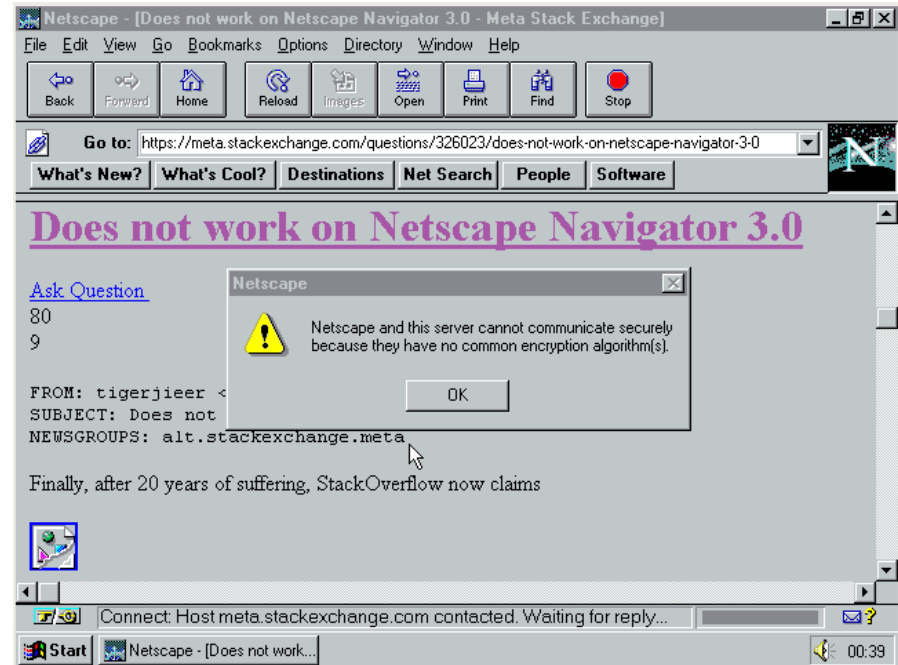


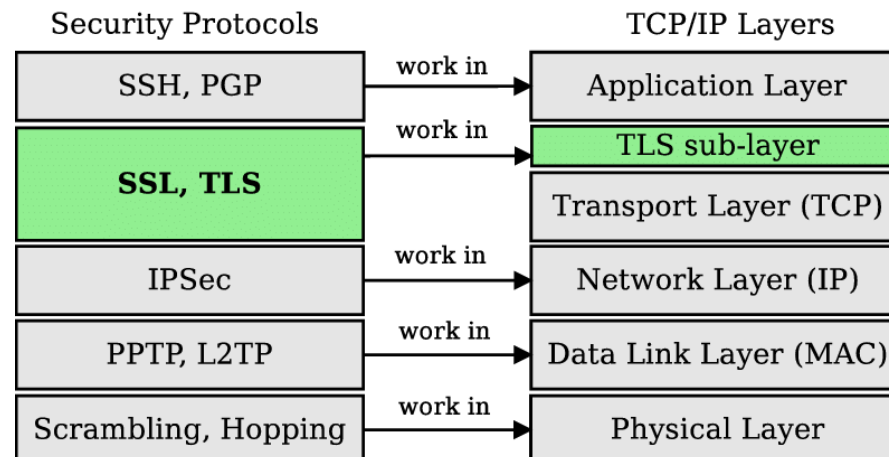
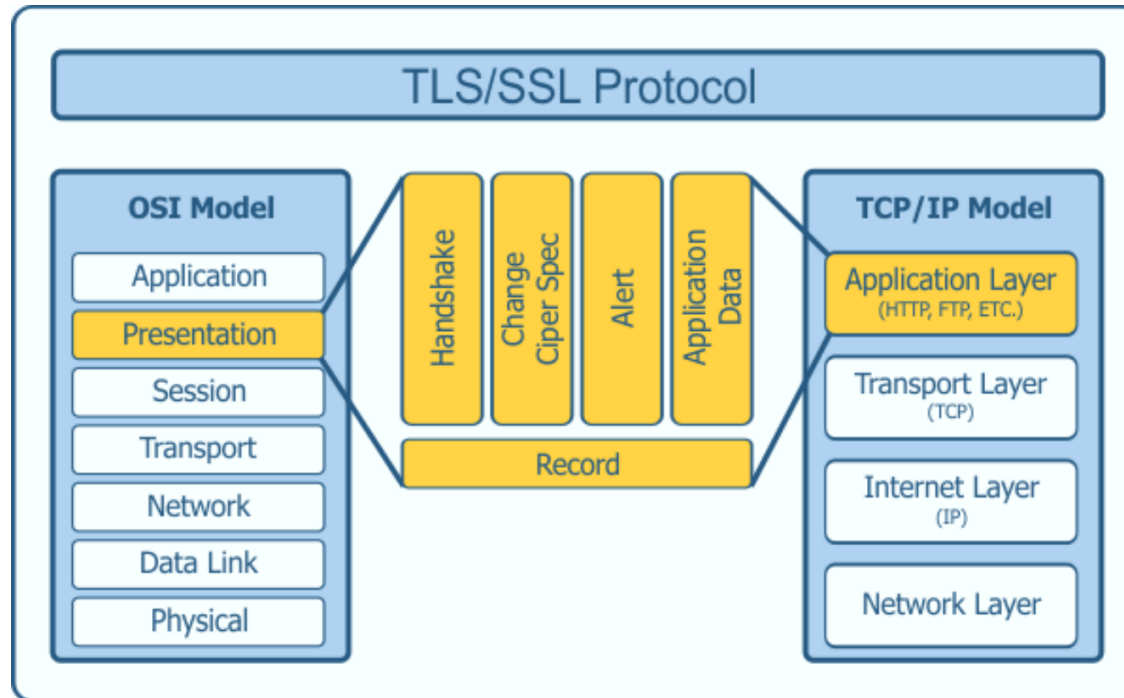
- <https://www.geeksforgeeks.org/how-does-certificate-based-authentication-work/>



TLS/SSL

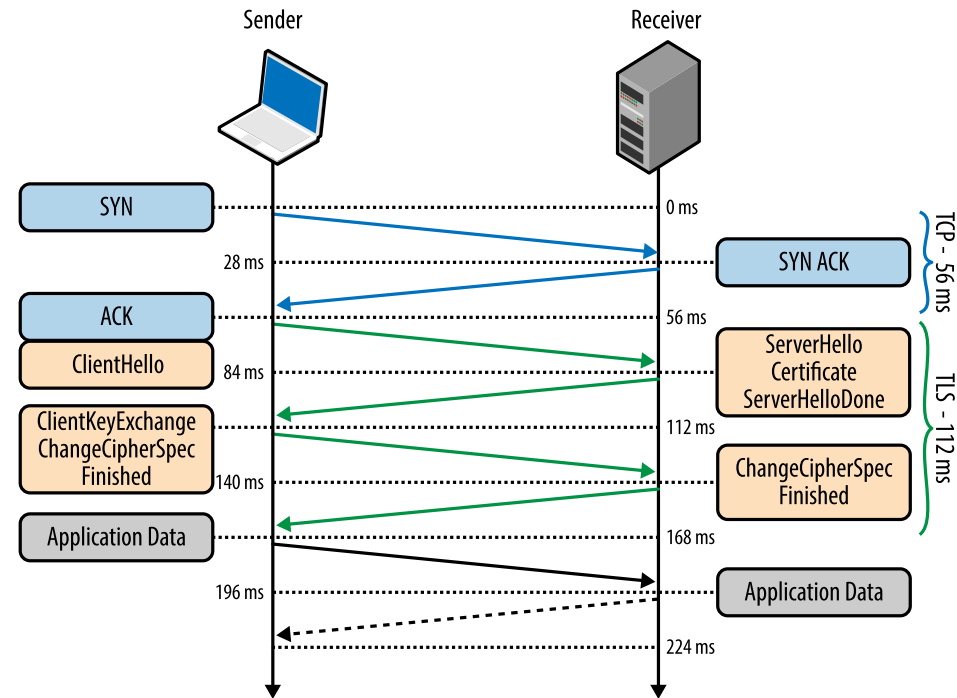
- TLS/SSL are cryptographic protocols for secure communication.
 - SSL (Secure Sockets Layer): Introduced by Netscape in 1995.
 - TLS (Transport Layer Security): Successor to SSL, standardized by IETF.
- *Purpose = good old CIA*





Year	Version	Key Features and Changes
1995	SSL 2.0	• First publicly released version by Netscape. • Vulnerable to various attacks (e.g., handshake weaknesses). • Deprecated in 2011.
1996	SSL 3.0	• Major improvements over SSL 2.0. • Introduced support for MAC (Message Authentication Codes). • Vulnerable to POODLE attack. • Deprecated in 2015.
1999	TLS 1.0 (RFC 2246)	• First version of TLS, replacing SSL 3.0. • Added HMAC for stronger integrity checks. • Vulnerable to BEAST attack. • Deprecated in 2021.
2006	TLS 1.1 (RFC 4346)	• Added protection against CBC (Cipher Block Chaining) attacks. • Improved handling of padding errors. • Deprecated in 2021.
2008	TLS 1.2 (RFC 5246)	• Support for SHA-2 hashing algorithms. • Allowed authenticated encryption with AES-GCM. • Made cipher suites more configurable. • Still widely used today.
2018	TLS 1.3 (RFC 8446)	• Major security improvements and performance enhancements. • Removed support for outdated algorithms (e.g., RSA key exchange, CBC mode). • Faster handshakes with 1-RTT (one round-trip time). • Forward secrecy is mandatory. • Zero Round Trip Time (0-RTT) for session resumption. • Stronger default cipher suites (e.g., AES-GCM, ChaCha20-Poly1305).

- Phase 1: TLS Handshake (Asymmetric Encryption)
 - Establishes a secure session and verifies identities via certificates
 - Negotiates encryption algorithms and keys
- Phase 2: Data Transmission (Symmetric Encryption)
 - Encrypts all data using a shared session key.
- Result: Secure communication with CIA



Message	Purpose	Contents
ClientHello	Initiates the handshake and proposes security settings.	• TLS Version • Random Number • Session ID (optional) • Cipher Suites • Extensions (SNI, ALPN, Supported Groups, Signature Algorithms)
ServerHello	Confirms security settings and provides server identity.	• TLS Version • Random Number • Selected Cipher Suite • Extensions (e.g., ALPN)
Server Certificate	Provides the server's identity for authentication.	• Digital Certificate (signed by CA) • Public Key • Certificate Chain (with intermediates) • CA Signature
Certificate Request (Optional)	Requests the client's certificate for mutual authentication.	• Certificate Types • CA Names (trusted authorities)
Client Certificate (Optional)	Provides the client's identity if requested by the server.	• Client Certificate • Public Key • Certificate Chain
Key Exchange (Pre-Master Secret)	Establishes a shared secret for session encryption.	• ECDHE Key Exchange (TLS 1.3) • Pre-Master Secret (Encrypted with Server's Public Key in TLS 1.2)
Certificate Verify (Optional)	Verifies the client's possession of the private key matching its certificate.	• Digital Signature (Signed hash of all handshake messages)
Finished Messages	Completes the handshake and verifies its integrity.	• Handshake Hash (Encrypted with session key) • Verifies handshake integrity

- TLS Cipher Suite Structure Example:

TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256

- TLS: Protocol
 - ECDHE: Key Exchange (Elliptic Curve Diffie-Hellman)
 - RSA: Authenticationz Algorithm
 - AES_128_GCM: Symmetric Encryption Algorithm
 - SHA256: Hashing Algorithm
-
- TLS improvements
 - only supports strong cipher suites.
 - removes outdated algorithms (e.g., RSA key exchange)

tcp.stream eq 7

No.	Time	Source	Destination	Prot	Len	Info
450	11:39:55...	hudson	cs282.wpc.edgecastcdn.net	TCP	74	58090 → 443 [SYN] Seq=3069499783 Win=29200 Len=0 MSS=1460 SACK_PERM=1...
453	11:39:55...	cs282.wpc.edgecast...	hudson	TCP	66	443 → 58090 [SYN, ACK] Seq=3490701116 Ack=3069499784 Win=65535 Len=0 ...
454	11:39:55...	hudson	cs282.wpc.edgecastcdn.net	TCP	54	58090 → 443 [ACK] Seq=3069499784 Ack=3490701117 Win=29312 Len=0
455	11:39:55...	hudson	cs282.wpc.edgecastcdn.net	TLSv1.2	261	Client Hello
462	11:39:55...	cs282.wpc.edgecast...	hudson	TCP	60	443 → 58090 [ACK] Seq=3490701117 Ack=3069499991 Win=147456 Len=0
463	11:39:55...	cs282.wpc.edgecast...	hudson	TLSv1.2	1514	Server Hello

Secure Sockets Layer

- TLSv1.2 Record Layer: Handshake Protocol: Client Hello
 - Content Type: Handshake (22)
 - Version: TLS 1.0 (0x0301)
 - Length: 202
 - Handshake Protocol: Client Hello
 - Handshake Type: Client Hello (1)
 - Length: 198
 - Version: TLS 1.2 (0x0303)
 - Random
 - Session ID Length: 0
 - Cipher Suites Length: 28
 - Cipher Suites (14 suites)
 - Cipher Suite: Unknown (0xfafa)
 - Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (0xc02b)
 - Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f)
 - Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 (0xc02c)
 - Cipher Suite: TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 (0xc030)
 - Cipher Suite: TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 (0xcca9)
 - Cipher Suite: TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256 (0xcca8)
 - Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (0xc013)
 - Cipher Suite: TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (0xc014)
 - Cipher Suite: TLS_RSA_WITH_AES_128_GCM_SHA256 (0x009c)
 - Cipher Suite: TLS_RSA_WITH_AES_256_GCM_SHA384 (0x009d)
 - Cipher Suite: TLS_RSA_WITH_AES_128_CBC_SHA (0x002f)
 - Cipher Suite: TLS_RSA_WITH_AES_256_CBC_SHA (0x0035)
 - Cipher Suite: TLS_RSA_WITH_3DES_EDE_CBC_SHA (0x000a)
 - Compression Methods Length: 1
 - Compression Methods (1 method)

```

0060 cb 00 00 1c fa fa c0 2b c0 2f c0 2c c0 30 cc a9 .....+././0..
0070 cc a8 c0 13 c0 14 00 9c 00 9d 00 2f 00 35 00 0a ...../5..
0080 01 00 00 81 ba ba 00 00 ff 01 00 01 00 00 00 00 .....
0090 18 00 16 00 00 13 63 64 6e 2d 62 65 74 61 2e 74 .....cd n-beta.t
00a0 75 6e 65 69 6e 2e 63 6f 6d 00 17 00 00 00 23 00 unein.co m....#.
00b0 00 00 0d 00 14 00 12 04 03 08 04 04 01 05 03 08 .....
  
```

List of cipher suites supported by client (ssl.handshake.ciphersuites), 28 bytes

Packets: 43448 · Displayed: 743 (1.7%) · Dropped: 0 (0.0%) · Profile: Default

- Common Threats
 - Expired Certificates: Cause service disruptions and warnings.
 - Man-in-the-Middle (MITM) Attacks: Interception of communication.
 - Cipher Suite Downgrade Attacks: Force use of weaker encryption.
 - Certificate Misuse: Stolen or fake certificates.
- Preventative Measures
 - Use HSTS (HTTP Strict Transport Security).
 - Enable `TLS_FALLBACK_SCSV` to prevent downgrades.
 - Implement Certificate Transparency Logs.
 - Use OCSP for real-time certificate status checking.

- Let's Encrypt is a **free, automated, and open** CA
- Launched in December 2015 by the Internet Security Research Group (ISRG)
 - <https://www.abetterinternet.org/>
- Provides SSL/TLS certificates to enable HTTPS and secure communication.
- Mission: “Encrypt the entire web” to promote a safer and more privacy-respecting internet.
 - Helps website owners secure their websites without cost or complexity.
 - Supported by: Major sponsors including Mozilla, Cisco, Google Chrome, Akamai, and Electronic Frontier Foundation (EFF).

1) Domain Validation (DV)

- Let's Encrypt provides DV certificates, verifying domain ownership through:
 - HTTP Challenge: Places a unique token on the website.
 - DNS Challenge: Adds a specific DNS TXT record.

2) ACME Protocol (Automatic Certificate Management Environment)

- ACME is an open protocol that automates the certificate issuance and renewal process.
- Users interact with ACME through ACME clients (e.g., Certbot).

3) Certificate Issuance

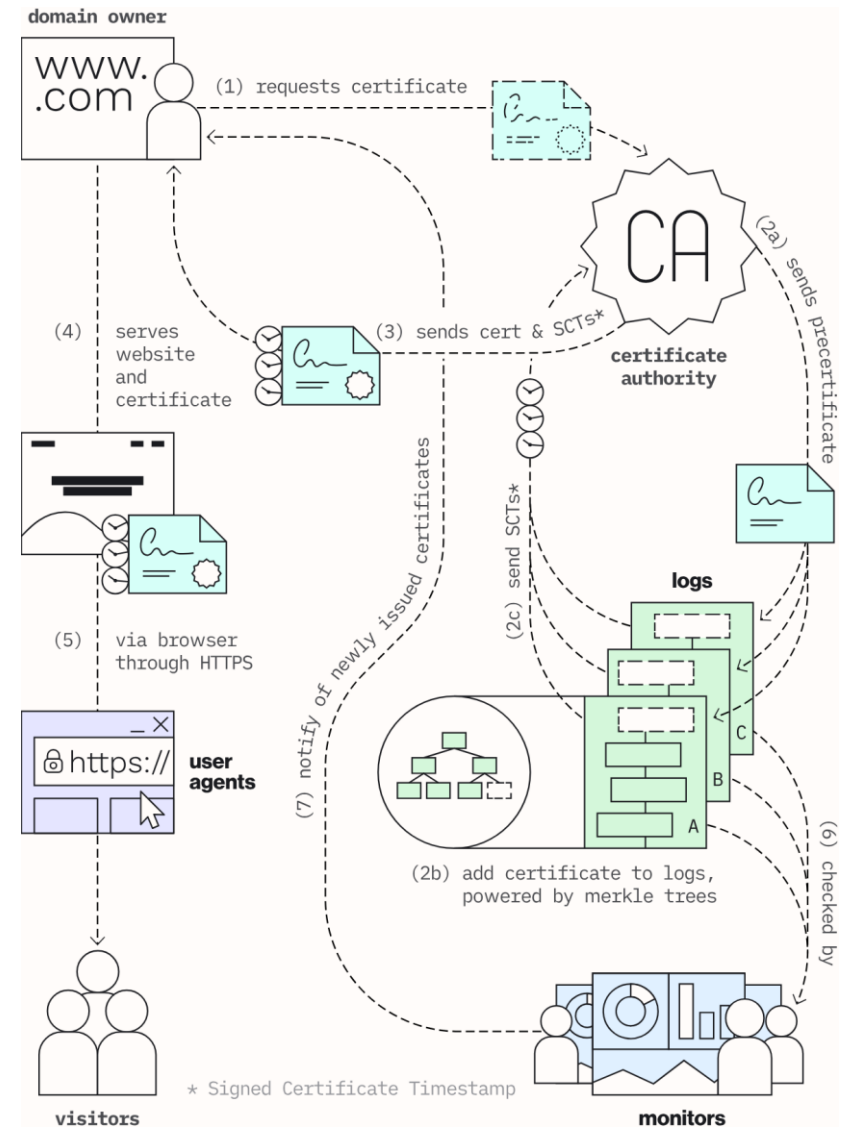
- Certificates are issued with a 90-day validity period for increased security.
- Can include wildcard certificates (e.g., *.example.com) using DNS validation.

4) Auto-Renewal

- Automatic renewal is achieved using ACME clients with cron jobs or systemd timers.
- ACME clients request a new certificate before expiration and update the web server automatically.

- **Certbot** (*Most Popular Client*)
 - developed by the Electronic Frontier Foundation (EFF).
 - supports automatic certificate installation for Apache, Nginx, etc.
- **acme.sh** (*CLI*)
 - a pure bash client with zero dependencies
 - ideal for custom deployments and Docker environments.
 - supports wildcard certificates with DNS validation.
- **Caddy** (*Web Server with Built-in HTTPS*)
 - A web server that automatically obtains and renews
 - requires zero configuration for HTTPS
- **Traefik** (*Reverse Proxy for Containers*)
 - ideal for microservices and Docker-based environments.
 - automatically handles Let's Encrypt integration and certificate management.
 - suitable for Kubernetes and cloud deployments.

- PKI is public, hence, can be monitored by anyone
- Monitor examples
 - <https://certificate.transparency.dev/monitors/>
 - <https://crt.sh/?id=6106822638>



- <https://www.ssllabs.com/ssl-pulse/>
- <https://www.ssllabs.com/ssltest/analyze.html?d=www.fit.vutbr.cz&s=147.229.9.23&latest>

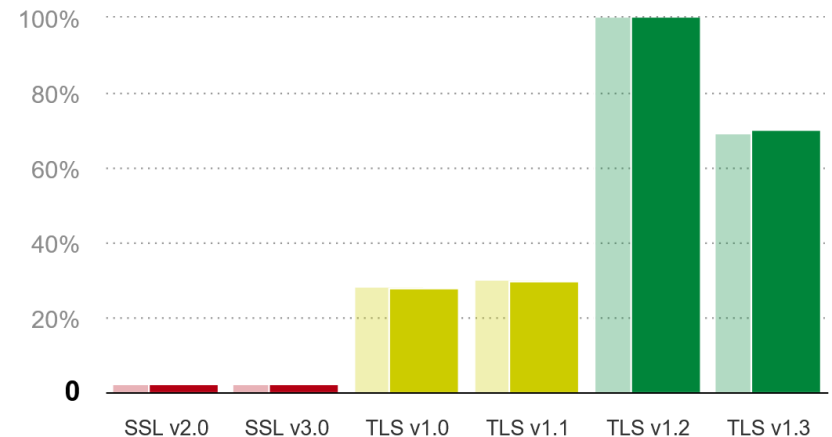
Strict Transport Security

47,863 Sites that support **HTTP Strict Transport Security**
 35.6 % of sites surveyed
 + 140 since previous month

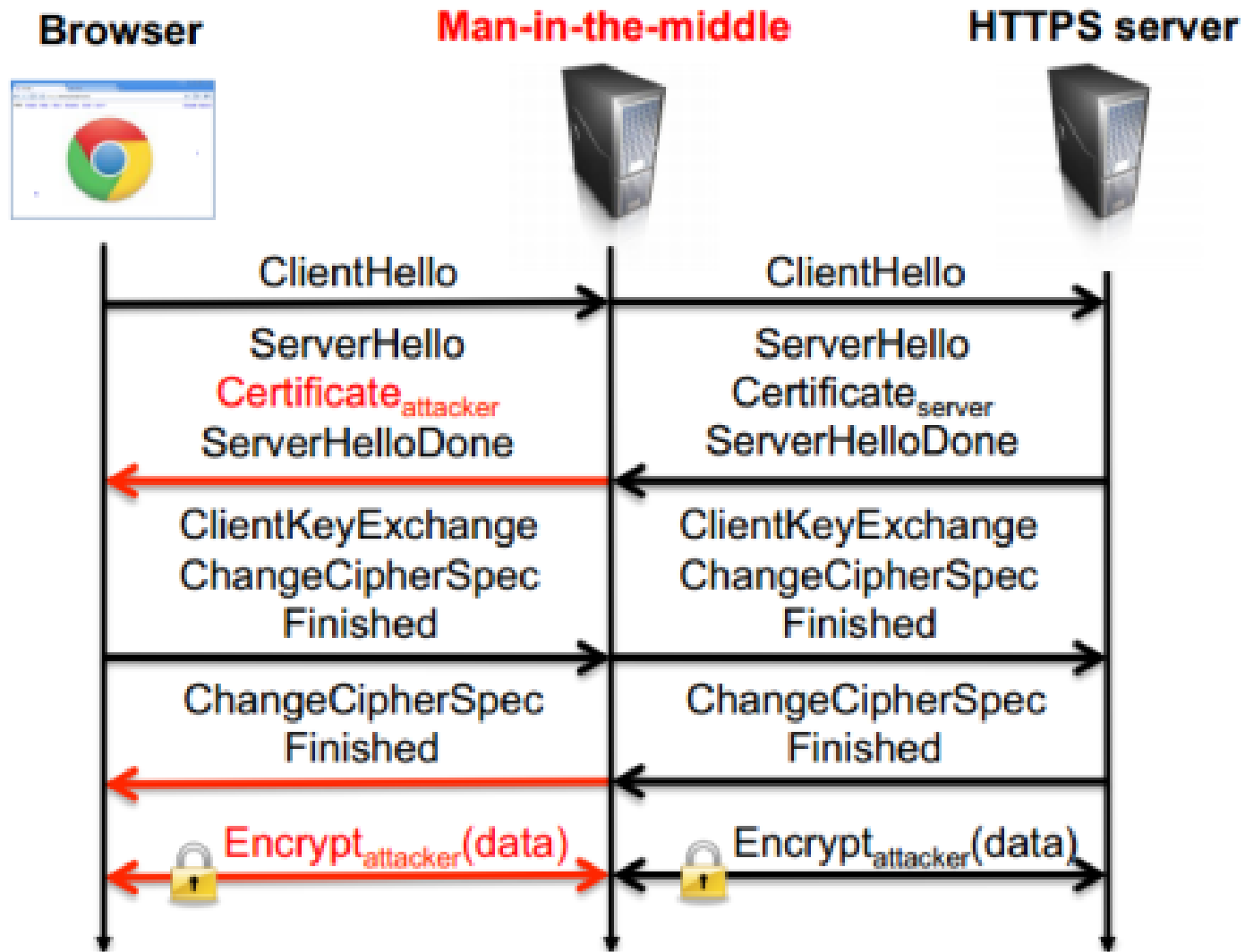
CAA

20,656 Sites that support **Certification Authority Authorization (RFC 6844)**
 15.4 % of sites surveyed
 + 56 since previous month

Protocol Support



HTTPS DECODING DEMO



- SSLSplit
 - https://github.com/nesfit/sslsplit_keylogger
 - <https://www.signal-chief.com/2022/02/implementing-sslsplit/>
- MITM Proxy
 - <https://www.petergirnius.com/blog/decrypting-https-traffic-with-mitmproxy-amp-wireshark>

- 1) Install Wireshark
- 2) Read about and install according to the preference
 - Hyper-V / VirtualBox / VMWare Workstation
 - Docker / LXC / Podman
 - GNS3, Eve-Ng, PNetLab
- 3) Reproduce demo with SSLsplit

Q & A

- What are properties of secure communication?
- What is authenticity and how is achieved?
- What is integrity and how is achieved?
- What is confidentiality and how is achieved?
- What is hashing function? Name examples and their properties.
- Describe symmetric-key algorithms. Name examples and their properties.
- Describe asymmetric-key algorithms. Name examples and their properties.
- What is PKI? Describe its components.
- What is certificate? What does it contain? Where is it used?
- What is TLS? On which layer it operates? What messages are being exchanges during TLS handshake?

- SCHWARTZ, Stu. *Cryptology for Beginners. Wissahickon HighAmbler, Pa*, 2004, 19002.
- AUMASSON, Jean-Philippe. *Serious cryptography: a practical introduction to modern encryption*. No Starch Press, Inc, 2024.
- MENEZES, Alfred J.; VAN OORSCHOT, Paul C.; VANSTONE, Scott A. *Handbook of applied cryptography*. CRC press, 2018.
- RISTIC, Ivan. *Bulletproof SSL and TLS: Understanding and deploying SSL/TLS and PKI to secure servers and web applications*. Feisty Duck, 2022.

Vladimír Veselý veselyv@fit.vut.cz



- <https://www.fit.vut.cz/research/groups/nes@fit/>
- <https://git.fit.vutbr.cz/NESFIT>
- <https://nesfit.github.io>